MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

TRANSFORMER-BASED TEXT NORMALIZATION IN VIETNAMESE SPEECH-TO-TEXT SYSTEMS

by

Bui The Viet

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

© Copyright by Bui The Viet 2021

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

TRANSFORMER-BASED TEXT NORMALIZATION IN VIETNAMESE SPEECH-TO-TEXT SYSTEMS

by

Bui The Viet

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

Supervisor:

Dr. Phan Duy Hung

© Copyright by Bui The Viet 2021

Transformer-based Text Normalization in Vietnamese Speech-to-text Systems

Bui The Viet

Degree Master of Software Engineering

FPT University

2021

Abstract

This thesis presents the task of normalizing Vietnamese transcribed texts in Speech-to-Text systems. The main purpose is to develop a text normalizer that automatically converts proper nouns and other context-specific formatting of the transcription such as dates, time, numbers, and punctuation marks into their appropriate expressions. To this end, this work proposes a solution that exploits deep neural networks with rich features followed by manually designed rules and transformer-based models to recognize and then convert these text sequences. It also introduces new Vietnamese pretrained language models to facilitate the process of the text normalization. The experimental results on this corpus are quite promising. Hope that this initial work will inspire other follow-up research on this important but unexplored problem.

Table of Contents

Table of Contents 2

List of Tables 4

List of Figures 5

Chapter 1 Introduction 7

1.1 Background 7

1.2 Focus of Research 7

Chapter 2 Empirical Studies on Vietnamese Transformer-based Models 9

2.1 Introduction 9

2.2 Bidirectional Encoder Representations from Transformers 11

2.3 Efficiently Learning an Encoder that Classifies Token Replacements Accurately 13

2.4 Experimental Settings 14

2.4.1 Model Training 14

2.4.2 Testing and evaluation methods 15

2.5 Experimental Results 16

2.5.1 Experimental Results on the PoS Tagging Tasks 16

2.5.2 Experimental Results on the NER Tasks 18

2.5.3 Decoding Time 20

2.6 Conclusion 21

Chapter 3 Spoken-form Entity Normalization with Rich Features 22

3.1 Introduction 22

3.2 Related Work 24

3.3 Neural Model Solutions with Rich Features 25

3.3.1 Problem Statement 25

3.3.2 A Proposed Solution to Normalize Texts 26

3.4 Experiments 28

3.4.1 Corpus Building 28

3.4.2 Experimental Setups 30

3.4.3 Experimental Results 30

3.5 Conclusion 34

Chapter 4 Punctuation Restoration using Transformer-based Models 35

4.1 Introduction 35

4.2 Related Work 37

4.3 A Proposed Model to Predict Punctuation 38

4.3.1 Problem Definition 38

4.3.2 A proposed architecture using pre-trained LMs with boundary context information 39

4.4 Experiments 42

4.4.1 Datasets 42

4.4.2 Experimental Setups 43

4.4.3 Evaluation Metrics 43

4.4.4 Experimental Results 44

4.4.5 Discussion 47

4.5 Conclusion 48

Chapter 5 Conclusion and Future Work 50

5.1 Conclusion 50

5.2 Future Work 50

References 51

List of Tables

Table 2.2.1. Information of Our proposed transformer-based architecture. 12

Table 2.5.1. Performance of our proposed models on the POS tagging tasks. 16

Table 2.5.2. Performance of our proposed models on NER tasks. 18

Table 3.4.1. Some statistics about the corpus. 29

Table 3.4.2. Experiment results using the rule-based baseline to detect entities. 31

Table 3.4.3. Experiment results using the neural architectures without rich features. 32

Table 3.4.4. Experiment results using the neural architectures with rich features. 32

Table 3.4.5. Some examples of wrong outputs of STT integrated with the text normalizer. 34

Table 4.4.1. Some statistics about the two Vietnamese benchmark datasets which are Novel and News. 42

Table 4.4.2. Experimental results on the NOVEL dataset WITHOUT using surrounding contexts. 45

Table 4.4.3. Experimental results on the NOVEL dataset using surrounding contexts..... 45

Table 4.4.4. Experimental results on the NEWSPAPER dataset WITHOUT surrounding contexts. 46

Table 4.4.5. Experimental results on the NEWSPAPER dataset using surrounding contexts. 46

Table 4.4.6. Experimental results of the best baseline method (biLSTM) and the best proposed model (vELECTRA) integrated with surrounding contexts (micro averaged on all 6 punctuation marks). 48

List of Figures

Figure 2.2.1. The basic structure of BERT. 11

Figure 2.2.2. Our proposed end-to-end architecture. 13

Figure 2.3.1. An overview of replaced token detection by the ELECTRA model on a sample drawn from vELECTRA. 14

Figure 2.5.1. F_1 scores of VLSP 2010 PoS Tagging. 17

Figure 2.5.2. F_1 scores of VLSP 2013 PoS Tagging. 17

Figure 2.5.3. F_1 scores of VLSP 2016 NER. 19

Figure 2.5.4. F_1 scores of VLSP 2018 NER. 19

Figure 2.5.5. Decoding time (in millisecond per sentence) on PoS task - VLSP 2013. 20

Figure 2.5.6. Decoding time (in millisecond per sentence) on NER task - VLSP 2016..... 20

Figure 3.1.1. An example of normalizing texts: one year, one person name, and a number are recognized and converted into their written formats. 23

Figure 3.3.1. An overall architecture of text normalization in STT system. 26

Figure 3.3.2. An architecture to detect entities in STT output using BiLSTMs. 28

Figure 3.4.1. The annotation process in building the new corpus. 32

Figure 3.4.2. Performance on each label in the F_1 scores (in %). 33

Figure 4.3.1. A proposed architecture using pre-trained LMs with the context information around each segment. 41

Figure 4.4.1. The F_1 scores using different training sizes using vELECTRA with context information on the NOVEL dataset. 47

Figure 4.4.2. The F_1 scores using different training sizes using vELECTRA with context information on the NEWSPAPER dataset. 47

Chapter 1
Introduction

1.1 Background

As the name would indicate, Speech-to-Text is a system that gets speech input and instantly generates texts as it is recognized from streaming audio or as the user is speaking. This type of automatic speech recognition systems generally produces un-normalized text which is difficult to read for humans and degrades the performance of many downstream machine processing tasks. Restoring the norm-texts greatly improves the readability of transcripts and increases the effectiveness of subsequent processing, like machine translation, summarization, question answering, sentiment analysis, syntactic parsing, and information extraction, etc.

Text normalization is a common pre-processing stage for a variety of speech and language processing applications, and its requirements vary greatly depending on the application-perhaps one reason why general machine learning methods to address the problem aren't as common as in other areas of the field. Tokenization and matching of tokens to an existing vocabulary are used in most text normalization approaches. Beyond that, the downstream application may need token mapping to new token sequences, including simple changes like de-casing to more sophisticated mappings, such as extending abbreviations or managing nonstandard spellings like those prevalent in social media.

Text normalization for voice applications, such as passing text via a Speech-to-Text engine to be recognized, is the subject of this study. This usage case differs from text normalization scenarios that do not include a spoken target since it has certain unique properties. For example, there are significant differences in how integers are handled. When normalizing text for syntactic parsing or other natural language algorithms that operate solely in the written domain.

1.2 Focus of Research

Acceptable latencies and error rates for text normalization for speech synthesis are dictated by application demands. Latency is an important concern since it is widely employed in mobile and voice assistant apps. Furthermore, certain mistakes are disastrous, affecting both the naturalness of the voice and the correctness of the rendering. Incorrectly inflecting number names in, instance,

Vietnamese, results in a lack of naturalness, although the outcome is passable in most circumstances. Normalizing transcribed texts, therefore, plays an important role in STT systems.

It usually consists of two main tasks:

(1) Punctuator detection which mainly focuses on periods (sentence boundaries).

(2) Automatically recognize and convert the spoken form of texts into their written expressions adhering to a single canonical rule.

In this thesis, we focus on the application of Vietnamese pretrained models to solve these two tasks. Firstly, chapter 2 shows our empirical studies on transformer-based models for Vietnamese language. Next, chapter 3 and 4 show our proposed solutions for the spoken-form entity normalization task and the punctuation prediction task. Finally, chapter 5 concludes our work.

Chapter 2
Empirical Studies on Vietnamese Transformer-based Models

Chapter 2 describes our study on using multilingual BERT embeddings and some new neural models for improving sequence tagging tasks for the Vietnamese language. We propose new model architectures and evaluate them extensively on two named entity recognition datasets of VLSP 2016 and VLSP 2018, and on two part-of-speech tagging datasets of VLSP 2010 and VLSP 2013. Our proposed models outperform existing methods and achieve new state-of-the-art results. We have pushed the accuracy of part-of-speech tagging to *95.40%* on the VLSP 2010 corpus, to *96.77%* on the VLSP 2013 corpus; and the F_1 score of named entity recognition to *94.07%* on the VLSP 2016 corpus, to *90.31%* on the VLSP 2018 corpus. Our code and pre-trained models viBERT and vELECTRA are released as open source to facilitate adoption and further research.

2.1 Introduction

Sequence modeling plays a central role in natural language processing. Many fundamental language processing tasks can be treated as sequence tagging problems, including part-of-speech tagging and named-entity recognition. In this chapter, we present our study on adapting and developing the multi-lingual BERT [1] and ELECTRA [2] models for improving Vietnamese part-of-speech tagging (PoS) and named entity recognition (NER). Many natural language processing tasks have been shown to be greatly benefited from large network pre-trained models. In recent years, these pre-trained models have led to a series of breakthroughs in language representation learning [3] [4] [5] [2]. Current state-of-the-art representation learning methods for language can be divided into two broad approaches, namely *denoising auto-encoders* and *replaced token detection*.

In the denoising auto-encoder approach, a small subset of tokens of the unlabeled input sequence, typically *15%*, is selected; these tokens are masked (e.g., BERT [1]), or attended (e.g., XLNet [5]); and then train the network to recover the original input. The network is mostly transformer-based models which learn bidirectional representation. The main disadvantage of these models is that they often require a substantial compute cost because only *15%* of the tokens per example is learned while a very large corpus is usually required for the pre-trained models to be effective. In the replaced token detection approach, the model learns to distinguish real input tokens from plausible but synthetically generated replacements (e.g., ELECTRA [2]). Instead of masking, this

method corrupts the input by replacing some tokens with samples from a proposal distribution. The network is pre-trained as a discriminator that predicts for every token whether it is an original or a replacement. The main advantage of this method is that the model can learn from all input tokens instead of just the small masked out subset. This is therefore much more efficient, requiring less than 1/4 of compute cost as compared to RoBERTa [6] and XLNet [5].

Both approaches belong to the fine-tuning method in natural language processing where we first pretrain a model architecture on a language modeling objective before fine-tuning that same model for a supervised downstream task. A major advantage of this method is that few parameters need to be learned from scratch. In this chapter, we propose some improvements over the recent transformer-based models to push the state-of-the-arts of two common sequence labeling tasks for Vietnamese. Our main contributions in this work are:

- We propose pre-trained language models for Vietnamese which are based on BERT and ELECTRA architectures; the models are trained on large corpora of 10GB and 60GB uncompressed Vietnamese text.

- We propose the fine-tuning methods by using attentional recurrent neural networks instead of the original fine-tuning with linear layers. This improvement helps improve the accuracy of sequence tagging.

- Our proposed system achieves new state-of-the-art results on all the four PoS tagging and NER tasks: achieving *95.04%* of accuracy on VLSP 2010, *96.77%* of accuracy on VLSP 2013, *94.07%* of F_1 score on NER 2016, and *90.31%* of F_1 score on NER 2018.

- We release code as open source to facilitate adoption and further research, including pretrained models viBERT and vELECTRA

The remainder of this chapter is structured as follows. Section 2.2 and 2.3 present the methods used in the current work. Section 2.4 and 2.5 describe the experimental results. Finally, Section 2.6 concludes the study and outlines some directions for future work.

2.2 Bidirectional Encoder Representations from Transformers

The basic structure of BERT [1] (***B**idirectional **E**ncoder **R**epresentations from **T**ransformers*) is summarized on Figure 2.2.1 where Trm are transformation and F_k are embeddings of the k-th token. In essence, BERT's model architecture is a multilayer bidirectional Transformer encoder based on the original implementation described in [7]. In this model, each input token of a sentence is represented by a sum of the corresponding token embedding, its segment embedding and its position embedding. The WordPiece embeddings are used; split word pieces are denoted by ##. In our experiments, we use learned positional embedding with supported sequence lengths up to 256 tokens.

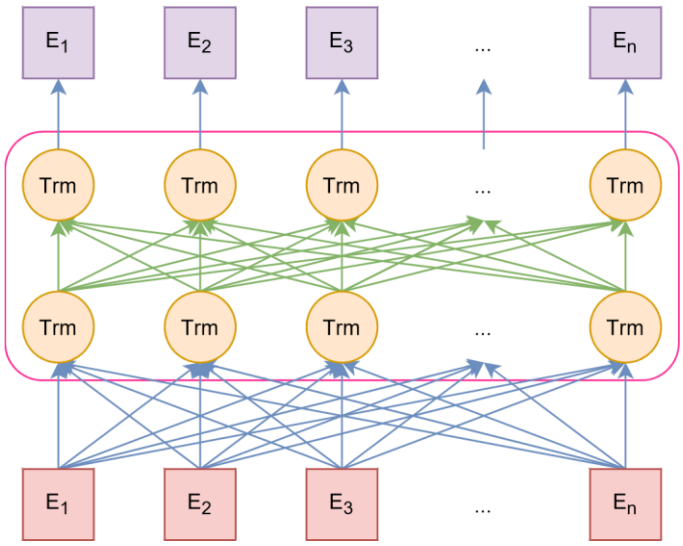


Figure 2.2.1. The basic structure of BERT.

The BERT model trains a deep bidirectional representation by masking some percentage of the input tokens at random and then predicting only those masked tokens. The final hidden vectors corresponding to the mask tokens are fed into an output SoftMax over the vocabulary. We use the whole word masking approach in this work. The masked language model objective is a cross entropy loss on predicting the masked tokens. BERT uniformly selects *15%* of the input tokens for masking. Of the selected tokens, *80%* are replaced with [MASK], *10%* are left unchanged, and *10%* are replaced by a randomly selected vocabulary token.

In our experiment, we start with the open source mBERT package[1]. We keep the standard hyperparameters of 12 layers, 768 hidden units, and 12 heads. The model is optimized with Adam [8] using the following parameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1e - 6$ and L_2 weight decay of 0.01.

The output of BERT is computed as follows [3]:

$$B_k = \gamma \left(w_0 E_k + \sum_{k=1}^{m} w_i h_{ki} \right)$$

where:

- B_k is the BERT output of k-th token.
- E_k is the embedding of k-th token.
- m is the number of hidden layers of BERT.
- h_{ki} is the i-th hidden state of k-th token.
- γ, w_1, w_2, ..., w_m are trainable parameters.

Our proposed architecture contains five main layers as follows of Table 2.2.1.

Layers	**Information**
Layer 1	The input layer encodes a sequence of tokens which are substrings of the input sentence, including ignored indices, padding and separators
Layer 2	A BERT layer
Layer 3	A bidirectional RNN layer with either LSTM or GRU units
Layer 4	An attention layer
Layer 5	A linear layer

Table 2.2.1. Information of Our proposed transformer-based architecture.

[1] https://github.com/google-research/ bert/blob/master/multilingual.md

A schematic view of our model architecture is shown in Figure 2.2.2.

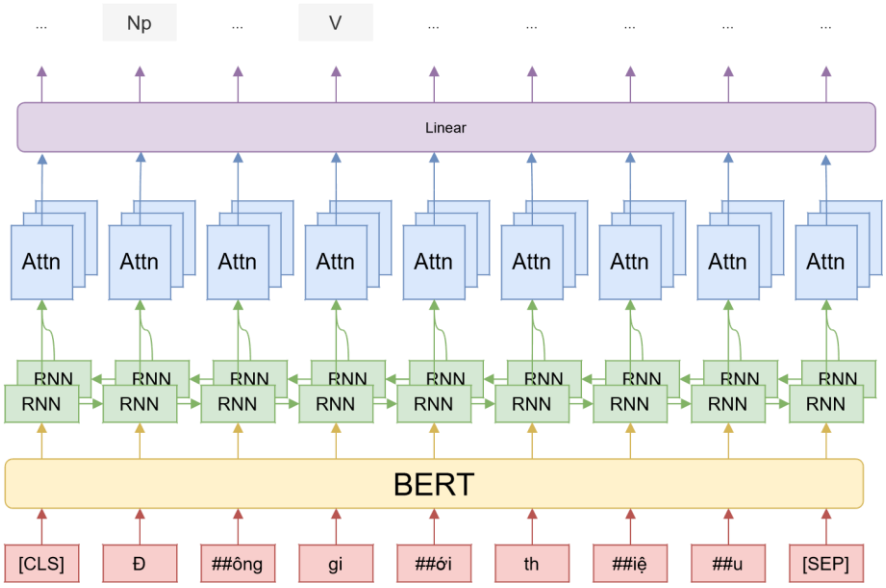


Figure 2.2.2. Our proposed end-to-end architecture.

2.3 Efficiently Learning an Encoder that Classifies Token Replacements Accurately

ELECTRA [2] (***E**fficiently **L**earning an **E**ncoder that **C**lassifies **T**oken **R**eplacements **A**ccurately*) is currently the latest development of BERT-based model where a more sample-efficient pretraining method is used. This method is called replaced token detection. In this method, two neural networks, a generator G and a discriminator D, are trained simultaneously. Each one consists of a Transformer network (an encoder) that maps a sequence of input tokens $\vec{x} = [x_1, x_2, \dots, x_n]$ into a sequence of contextualized vectors $h(\vec{x}) = [h_1, h_2, \dots, h_n]$. For a given position t where x_t is the masked token, the generator outputs a probability for generating a particular token x_t with a SoftMax distribution:

$$p_G(x_t|\vec{x}) = \frac{exp(x_t^T h_G(\vec{x})_t)}{\sum_u exp(u_t^T h_G(\vec{x})_t)}$$

For a given position t, the discriminator predicts whether the token x_n is 'real' i.e., that it comes from the data rather than the generator distribution, with a sigmoid function:

$$D(\vec{x}, t) = \sigma(w^T h_D(\vec{x})_t)$$

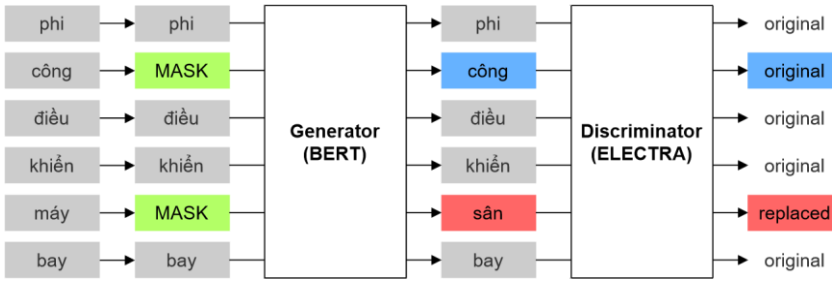


Figure 2.3.1. An overview of replaced token detection by the ELECTRA model on a sample drawn from vELECTRA.

An overview of the replaced token detection in the ELECTRA model is shown in Figure 2.3.1. The generator is a BERT model which is trained jointly with the discriminator. The Vietnamese example is a real one which is sampled from our training corpus.

2.4 Experimental Settings

2.4.1 Model Training

To train the proposed models, we use a CPU (Intel Xeon E5-2699 v4 @ 2.20GHz) and a GPU (NVIDIA GeForce GTX 1080 Ti 11G). The hyper-parameters that we chose are as follows: maximum sequence length is 256, BERT learning rate is $2e$-5, learning rate is $1e$-3, number of epochs is 100, batch size is 16, use apex and BERT weight decay is set to 0, the Adam rate is $1e$-8. The configuration of our model is as follows: number of RNN hidden units is 256, one RNN layer, attention hidden dimension is 64, number of attention heads is 3 and a dropout rate of 0.5.

To build the pre-training language model, it is very important to have a good and big dataset. This dataset was collected from online newspapers[2] in Vietnamese. To clean the data, we perform the following pre-processing steps:

- Remove duplicated news
- Only accept valid letters in Vietnamese

[2] vnexpress.net, dantri.com.vn, baomoi.com, zingnews.vn, vitalk.vn, etc.

- Remove too short sentences (less than 4 words)

We obtained approximately 10GB of texts after collection. This dataset was used to further pretrain the mBERT to build our viBERT which better represents Vietnamese texts. About the vocab, we removed insufficient vocab from mBERT because its vocab contains ones for other languages. This was done by keeping only vocabs existed in the dataset.

In pre-training vELECTRA, we collect more data from two sources:

- NewsCorpus: 27.4 GB[3]
- OscarCorpus: 31.0 GB[4]

Totally, with more than 60GB of texts, we start training different versions of vELECTRA. It is worth noting that pre-training viBERT is much slower than pre-training vELECTRA. For this reason, we pre-trained viBERT on the 10GB corpus rather than on the large 60GB corpus.

2.4.2 Testing and evaluation methods

In performing experiments, for datasets without development sets, we randomly selected 10% for fine-tuning the best parameters.

To evaluate the effectiveness of the models, we use the commonly used metrics which are proposed by the organizers of VLSP. Specifically, we measure the accuracy score on the POS tagging task which is calculated as follows:

$$Acc = \frac{\#of_words_correcly_tagged}{\#of_words_in_the_test_set}$$

and the F_1 score on the NER task using the following equations:

$$F_1 = 2 \cdot \frac{Pre \cdot Rec}{Pre + Rec}$$

[3] https://github.com/binhvq/news-corpus

[4] https://traces1.inria.fr/oscar/

where *Pre* and *Rec* are determined as follows:

$$Pre = \frac{NE_{true}}{NE_{sys}}$$

$$Rec = \frac{NE_{true}}{NE_{ref}}$$

where NE_{ref} is the number of NEs in gold data, NE_{sys} is the number of NEs in recognizing system, and NE_{true} is the number of NEs which is correctly recognized by the system.

2.5 Experimental Results

2.5.1 Experimental Results on the PoS Tagging Tasks

Table 2.5.1. Performance of our proposed models on the POS tagging tasks. shows experimental results using different proposed architectures on the top of mBERT and viBERT and vELECTRA on two benchmark datasets from the campaign VLSP 2010 and VLSP 2013.

No.		**VLSP 2010**			**VLSP 2013**		
Existing models							
1.		MEM [9]		93.4	RDRPOSTagger [10]		95.1
2.					BiLSTM-CNN-CRF [11]		95.4
3.					VnCoreNLP-POS [12]		95.9
4.					jointWPD [13]		96.0
5.					PhoBERT_base [14]		96.7
Proposed models							
	Model name	**mBERT**	**viBERT**	**vELEC**	**mBERT**	**viBERT**	**vELEC**
1.	+Fine-Tune	94.34	95.07	95.35	96.35	96.60	96.62
2.	+BiLSTM	94.34	95.12	95.32	96.38	96.63	**96.77**
3.	+BiGRU	94.37	95.13	95.37	96.45	96.68	96.73
4.	+BiLSTM_Attn	94.37	95.12	**95.40**	96.36	96.61	96.61
5.	+BiGRU_Attn	94.41	95.13	95.35	96.33	96.56	96.55

Table 2.5.1. Performance of our proposed models on the POS tagging tasks.

As can be seen that, with further pre-training techniques on a Vietnamese dataset, we could significantly improve the performance of the model. On the dataset of VLSP 2010, both viBERT and vELECTRA significantly improved the performance by about *1%* in the F_1 scores. On the dataset of VLSP 2013, these two models slightly improved the performance.

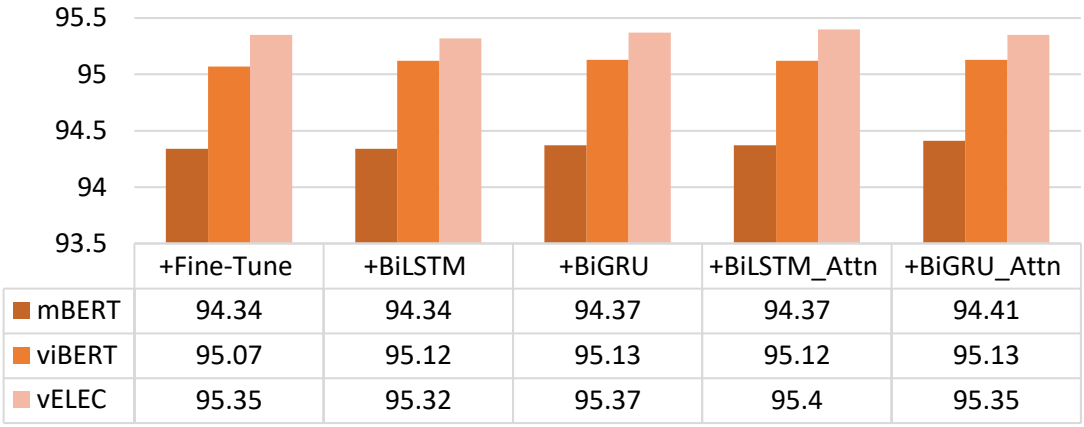


Figure 2.5.1. F_1 scores of VLSP 2010 PoS Tagging.

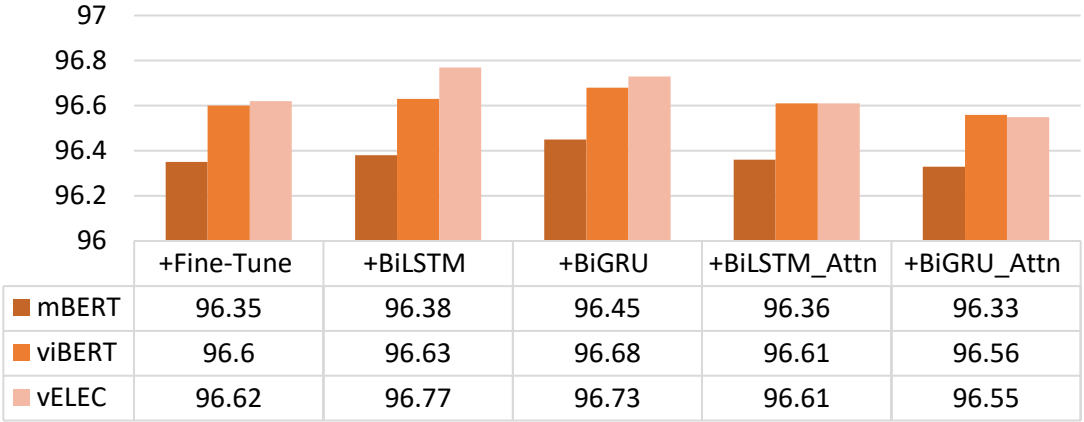


Figure 2.5.2. F_1 scores of VLSP 2013 PoS Tagging.

From the table, we can also see the performance of different architectures including fine-tuning, BiLSTM, BiGRU, and their combination with attention mechanisms. Figure 2.5.1 and Figure 2.5.2 show the benchmark scores of VLSP 2010 PoS Tagging task and VLSP 2013 Pos Tagging respectively. Fine-tuning mBERT with linear functions in several epochs could produce nearly state-of-the-art results. It is also shown that building different architectures on top slightly improve the performance of all mBERT, viBERT and vELECTRA models. On the VLSP 2010, we got the

accuracy of *95.40%* using biLSTM with attention on top of vELECTRA. On the VLSP 2013 dataset, we got *96.77%* in the accuracy scores using only biLSTM on top of vELECTRA.

In comparison to previous work, our proposed model - vELECTRA - outperformed previous ones. It achieved from *1%* to *2%* higher than existing work using different innovation in deep learning such as CNN, LSTM, and joint learning techniques. Moreover, vELECTRA also gained a slightly better than PhoBERT_base, the same pre-training language model released so far, by nearly *0.1%* in the accuracy score.

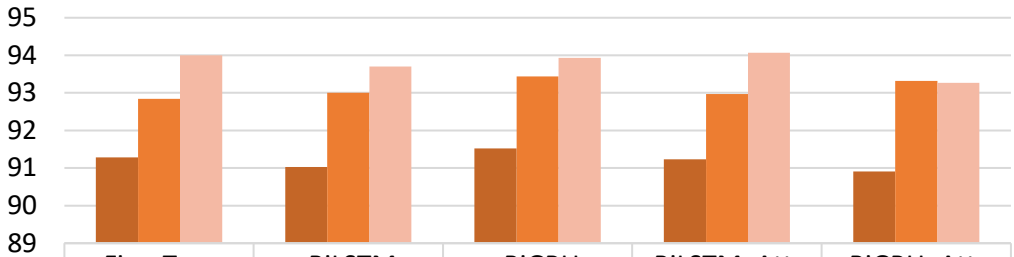
2.5.2 Experimental Results on the NER Tasks

Table 2.5.2 shows experimental results using different proposed architectures on the top of mBERT, viBERT and vELECTRA on two benchmark datasets from the campaign VLSP 2016 and VLSP 2018.

No.		**VLSP 2016**			**VLSP 2018**		
Existing models							
1.		TRE+BI [15]		87.98	ZA-NER [16]		74.70
2.		BiLSTM_CNN_CRF [17]		88.59	VietNER [18] [19]		76.63
3.		BiLSTM [20]		92.02			
4.		NNVLP [21]		92.91			
5.		VnCoreNLP-NER [12]		88.60			
6.		VNER [22]		89.60			
7.		ETNLP [23]		91.10			
8.		PhoBERT_base [14]		93.60			
Proposed models							
	Model name	**mBERT**	**viBERT**	**vELEC**	**mBERT**	**viBERT**	**vELEC**
1.	+Fine-Tune	91.28	92.84	94.00	86.86	88.04	89.79
2.	+BiLSTM	91.03	93.00	93.70	86.62	88.68	89.92
3.	+BiGRU	91.52	93.44	93.93	86.72	88.98	**90.31**
4.	+BiLSTM_Attn	91.23	92.97	**94.07**	87.12	89.12	90.26
5.	+BiGRU_Attn	90.91	93.32	93.27	86.33	88.59	89.94

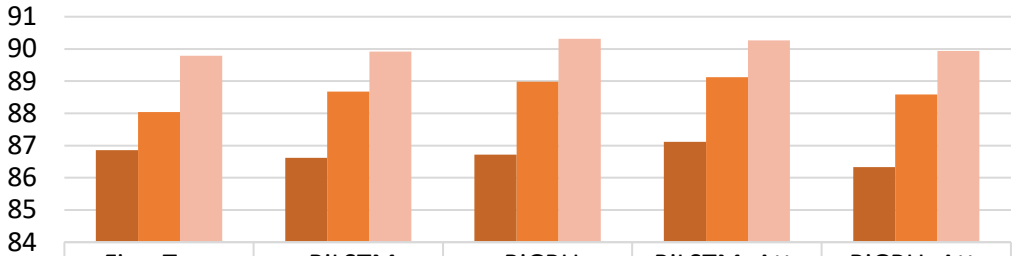
Table 2.5.2. Performance of our proposed models on NER tasks.

These results once again gave strong evidence to the above statement that further training mBERT on a small raw dataset could significantly improve the performance of transformation-based language models on downstream tasks. Training vELECTRA from scratch on a big Vietnamese dataset could further enhance the performance. On two datasets, vELECTRA improve the F_1 score by from *1%* to *3%* in comparison to viBERT and mBERT.



	+Fine-Tune	+BiLSTM	+BiGRU	+BiLSTM_Attn	+BiGRU_Attn
mBERT	91.28	91.03	91.52	91.23	90.91
viBERT	92.84	93	93.44	92.97	93.32
vELEC	94	93.7	93.93	94.07	93.27

Figure 2.5.3. F_1 scores of VLSP 2016 NER.



	+Fine-Tune	+BiLSTM	+BiGRU	+BiLSTM_Attn	+BiGRU_Attn
mBERT	86.86	86.62	86.72	87.12	86.33
viBERT	88.04	88.68	88.98	89.12	88.59
vELEC	89.79	89.92	90.31	90.26	89.94

Figure 2.5.4. F_1 scores of VLSP 2018 NER.

The benchmark results for the VLSP 2016 and the VLSP 2013 are shown in Figure 2.5.3 and Figure 2.5.4. Looking at the performance of different architectures on top of these pre-trained models, we acknowledged that biLSTM with attention once again yielded the SOTA result on VLSP 2016 dataset. On VLSP 2018 dataset, the architecture of BiGRU yielded the best performance at *90.31%* in the F_1 score.

Comparing to previous work, the best proposed model outperformed all work by a large margin on both datasets.

2.5.3 Decoding Time

Figure 2.5.5 and Figure 2.5.6 show the averaged decoding time measured on one sentence. According to our statistics, the average length of one sentence in VLSP 2013 and VLSP 2016 datasets are 22.55 and 21.87 words, respectively.

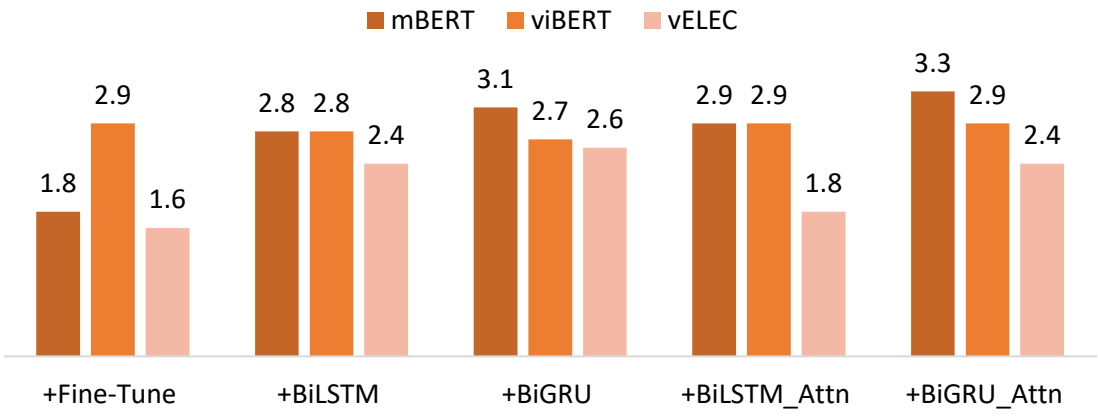


Figure 2.5.5. Decoding time (in millisecond per sentence) on PoS task - VLSP 2013.

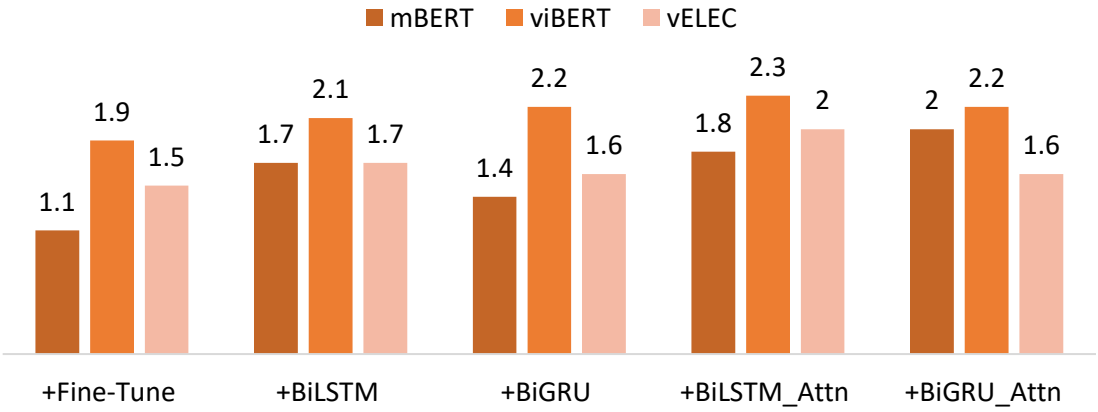


Figure 2.5.6. Decoding time (in millisecond per sentence) on NER task - VLSP 2016

For the POS tagging task measured on VLSP 2013 dataset, among three models, the fastest decoding time is of vELECTRA model, followed by viBERT model, and finally by mBERT model. This statement holds for four proposed architectures on top of these three models.

However, for the fine-tuning technique, the decoding time of mBERT is faster than that of viBERT.

For the NER task measured on the VLSP 2016 dataset, among three models, the slowest time is of viBERT model with more than 2 milliseconds per sentence. The decoding times on mBERT topped with simple fine-tuning techniques, or bi-GRU, or biLSTM-attention is a little bit faster than on vELECTRA with the same architecture.

This experiment shows that our proposed models are of practical use. In fact, they are currently deployed as a core component of our commercial chatbot engine FPT.AI which is effectively serving many customers. More precisely, the FPT.AI platform has been used by about 70 large enterprises, and of over 27,000 frequent developers, serving more than 30 million end users.[5]

2.6 Conclusion

This chapter presents some new model architectures for sequence tagging and our experimental results for Vietnamese part-of-speech tagging and named entity recognition. Our proposed model vELECTRA outperforms previous ones. For part-of-speech tagging, it improves about *2%* of absolute point in comparison with existing work which use different innovation in deep learning such as CNN, LSTM, or joint learning techniques. For named entity recognition, the vELECTRA outperforms all previous work by a large margin on both VLSP 2016 and VLSP 2018 datasets.

Our code and pre-trained models are published as an open-source project for facilitate adoption and further research in the Vietnamese language processing community. An online service of the models for demonstration is also accessible at https://fpt.ai/nlp/bert/. A variant and more advanced version of this model is currently deployed as a core component of our commercial chatbot engine FPT.AI which is serving effectively millions of end users. These models are being fine-tuned to improve task-oriented dialogue in mixed and multiple domains [24] and dependency parsing [25]. This chapter presents some new model architectures.

[5] These numbers are reported as of August 2020

Chapter 3
Spoken-form Entity Normalization with Rich Features

Chapter 3 presents the task of normalizing Vietnamese transcribed texts in Speech-to-Text (STT) systems. The main purpose is to develop a text normalizer that automatically converts proper nouns and other context-specific formatting of the transcription such as dates, time, and numbers into their appropriate expressions. To this end, we propose a solution that exploits deep neural networks with rich features followed by manually designed rules to recognize and then convert these text sequences. We also introduce a new corpus of 13K spoken sentences to facilitate the process of the text normalization. The experimental results on this corpus are quite promising. The proposed method yields *90.67%* in the F_1 score in recognizing sequences of texts that need converting. We hope that this initial work will inspire other follow-up research on this important but unexplored problem.

3.1 Introduction

As the name would indicate, Speech-to-Text is a system that gets speech input and instantly generates texts as it is recognized from streaming audio or as the user is speaking. This type of automatic speech recognition systems generally produces un-normalized text (as indicated in Figure 3.1.1) which is difficult to read for humans and degrades the performance of many downstream machine processing tasks. Restoring the norm-texts greatly improves the readability of transcripts and increases the effectiveness of subsequent processing, like machine translation, summarization, question answering, sentiment analysis, syntactic parsing, and information extraction, etc. Normalizing transcribed texts, therefore, plays an important role in STT systems.

It usually consists of two main tasks:

(1) Punctuator detection which mainly focuses on periods (sentence boundaries).

(2) Automatically recognize and convert the spoken form of texts into their written expressions adhering to a single canonical rule.

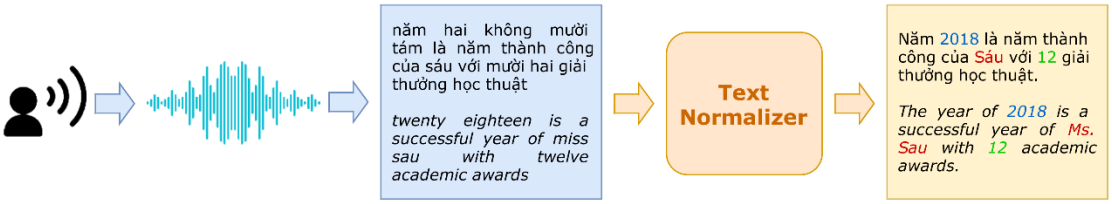


Figure 3.1.1. An example of normalizing texts: one year, one person name, and a number are recognized and converted into their written formats.

In this study, we assume that the former task is solved by using a simple technique based on the information of long silence between speeches, to identify sentence boundary. We, hence, concentrate on the latter task which aims to automatically transcribe proper nouns and typical context-specific formatting. Three main types of proper nouns which are person names, organization names, and location names; and three typical context-specific formatting such as dates, time, and numbers are considered in this research. Such proper nouns and formatting are called entities in this chapter.

General speaking, detecting such entities requires quite a bit of linguistic sophistication and native speaker intuition [26]. In the running example of Figure 3.1.1, the first occurrence of ‘year’ is ambiguous with the digit ‘five’, or the person name ‘Sáu’ is ambiguous with the number ‘six’ if their surrounding contexts are not fully considered. Hence, automatically disambiguating these cases is a challenging task because of different ambiguity issues existing in both un-normal texts and other forms of texts.

To our knowledge, it seems that there has no related public research on text normalization in spoken forms of texts so far. Hence, this chapter is the first work that formulates the task and provides a preliminary solution to solve it. To this end, the proposed solution is a hybrid architecture that exploits deep neural networks with rich features followed by rule-based processing. Specifically, LSTM [27] [28] and CNN [29] models integrated with rich manually built features are used to automatically detect these entities. Then, some necessary rules are built to convert these detected entities into their appropriate written expressions. A new corpus consisting of 13K spoken sentences is also annotated to allow deep learning solutions to be deployed. In conclusion, this chapter makes the following contributions:

- Presents the new task of normalizing texts in Vietnamese STT systems.

- Provides a preliminary solution based on neural networks with rich features followed by rules.
- Introduce a new corpus to conduct experiments and facilitate the process of normalizing texts in STT systems.

The rest of this chapter is organized as follows. The next section discusses related work. Then, we formally define the problem, and propose a solution to solve it. After that, we introduce our new manually built corpus in a general domain and shows some statistics. Experimental setups, experimental results, and some discussions are also reported. Finally, we conclude the paper and point out some future lines of work.

3.2 Related Work

Text normalization is an important stage in processing non-canonical language from natural sources such as social texts, speech, short messages, etc. This is a new research field and most of its papers published are done for popular languages such as English, Japanese, Chinese, etc. All these text normalization systems usually focus on social texts [30] [31] [32], short messages [33], text-to-speech systems [34], etc. For example, Eryigit and Torunoglu-selamet [30] present the first work on the social media text normalization of an MRL and introduces the first text normalization system for Turkish. Ikeda, Shindo, and Matsumoto [31] present a Japanese text normalization using Encoder-Decoder model. Aw et al. [33] propose a phrase-based statistical model for normalizing SMS texts. For text normalization systems involving speech and language technologies, there have been several works to convert texts from written expressions into their appropriate 'spoken' forms. For example, Yolchuyeva, Gyires-Toth, and Nemeth [34] introduce a novel CNNs based text normalizer and verify its effectiveness on the dataset of a text normalization challenge on Kaggle.[6]

To our knowledge, there is no public research focusing on the spoken forms of texts in STT systems, especially in Vietnamese. Spoken forms of texts behave quite differently from normal written texts and have some very special phenomena. They are much longer and highly ambiguous (as can be seen in the above examples). To normalize the spoken texts, a straightforward approach

[6] https://www.kaggle.com/c/text-normalization-challenge-english-language

is to use predefined rules because they seem to follow some under-lying syntactical patterns. These rules can be designed by observing the output of STT systems. However, the approach still poses some disadvantages such as difficult to construct highly accurate rules, time-consuming, need domain-expert skills, difficult to maintain and extend rules, and not effective.

This is since the rule-based approach usually could not deal well with ambiguity problems. To a large extent, it is necessary to consider semantic information of texts and their surrounding contexts. Rather than using rules, this chapter proposes a machine learning-based architecture to solve the task. This approach exploits deep neural networks with rich features followed by some language-specific heuristic rules to recognize and convert text sequences need normalizing into their right formats.

3.3 Neural Model Solutions with Rich Features

In this section, we first formally state the problem and then propose a solution to address it.

3.3.1 Problem Statement

The problem can be stated as follows: Given a sequence of syllables which are outputs of an STT system $S = \{s_1, s_2, ..., s_n\}$, s_i is the ith syllable (assuming that the output was sentence segmented), it is required to transcribe S into clean verbatim text formats as follows:

- Capitalize the first letter of the first syllable s_1 in S
- Capitalize the first letter of each syllable s_k in any proper noun. For our analysis, we consider three common types of proper nouns which are person names, organization names, and location names.
- Capitalize all letters in a syllable s_k if s_k is a course identifier (e.g., VN247) or an abbreviation of organization names (e.g., WHO, FPT, VNPT, etc.) or an abbreviation of organization names (e.g., WHO, FPT, VNPT, etc.).
- Write out numbers zero through ten unless they are part of some cases such as sports records (2-0), time, binary, date, etc. Numbers above 10 represent numerical digits. For numbers above 999.999, substitute million, billion, etc., for the zeros. For dates/time, we use the Vietnamese formats of *dd/MM/yyyy* and *hh:mm:ss*.

Replace uoms (unit-of-measurements) with their symbols such as Hz, %, USD, etc.

3.3.2 A Proposed Solution to Normalize Texts

The overall architecture is presented in Figure 3.3.1 with three main steps as follows:

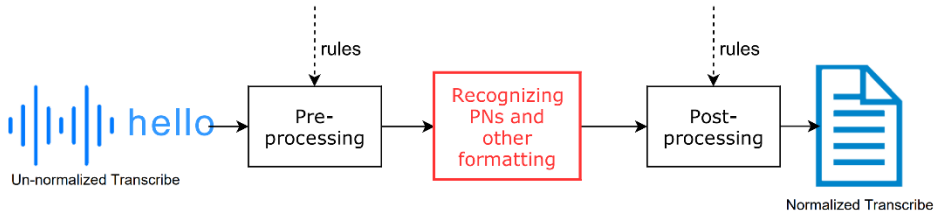


Figure 3.3.1. An overall architecture of text normalization in STT system.

(1) Pre-processing: The texts are pre-processed to capture phone numbers, URLs, e-mail address, ... if they follow their formal syntactical patterns.

(2) Recognizing Proper Nouns and other text formatting: This is the most critical and difficult step. To automatically recognize if segments of texts are currently considered as valid entities, it is necessary to know its surrounding contexts. In this work, instead of using heuristic rules, we exploit machine learning techniques by modeling the task as a sequence labeling problem. A fast and effective strategy to label each word is to use its own features to predict labels independently. The best solution is to make the optimal label for a given element dependent on the choices of nearby elements. To this end, we use CRFs [35] which are widely applied and yield state-of-the-art results in many NLP problems [36] [37].

(3) Post-processing: After detected, this step applies some rules to convert these entities into their written expressions as follows:

(a) If a syllable belongs to any recognized proper nouns, just capitalize its first letter. We do a further step to check if this syllable appears in a pre-defined list of course identifiers; we just capitalize every single character of it.

(b) If a sequence of syllables is determined as a date, time, or a number, we built corresponding rules to convert it into its written form. To design these rules, we ask the help of two linguistic experts to write down all possible reading methods/styles in different regions of Vietnam. Then, rules are gradually composed to catch up almost these possible methods.

(c) For uom, a list is manually built to help converting them into their correct symbols.

To build a strong model, CRFs need a good feature set. These features will be automatically learned via neural models and then enriched with manual-built features m. Figure 3.3.2 shows the architecture of applying neural architectures to automatically extract useful features for the model. We first use convolutional neural networks by [29] [38] to encode character-level information of a tth word into its character-level representation l_t. l_t was initialized randomly and trained with the whole network of CNNs. We then combine l_t with word-level representations w_t and manually built features m_t feed $x_t = concat(l_t, w_t, m_t)$ into bi-LSTM networks [28] [39] to model context information of each word. Formally, the formulas to update an LSTM unit at time t are:

$$
\begin{aligned}
i_t &= \sigma(W_i h_{t-1} + U_i X_t + b_i) \\
f_t &= \sigma\left(W_f h_{t-1} + U_f X_t + b_f\right) \\
\bar{c}_t &= tanh(W_c h_{t-1} + U_c X_t + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \bar{c}_t \\
o_t &= \sigma(W_0 h_{t-1} + U_0 X_t + b_0) \\
h_t &= o_t \odot tanh(c_t)
\end{aligned}
$$

Where:

- σ is the element-wise sigmoid function and $\odot$ is the element-wise product.
- x_t is input the vector (concatenation of character, word, and manual rich features embeddings) at time t.
- h_t is the hidden state vector which stores all the useful information up to time t.
- U_i, U_f, U_c, and U_o denote the weight matrices of different gates for input x_t.
- W_i, W_f, W_c, and W_o are the weight matrices for hidden state h_t.
- b_i, b_f, b_c, and b_o denote the bias vectors.

The idea of using bi-LSTMs is to present each sequence forwards and backward to capture past and future information, respectively. These two hidden states l_t and r_t are then concatenated to form the final output. Finally, a CRF is used to consider neighboring tags, yielding the final context predictions for every word of the input texts.

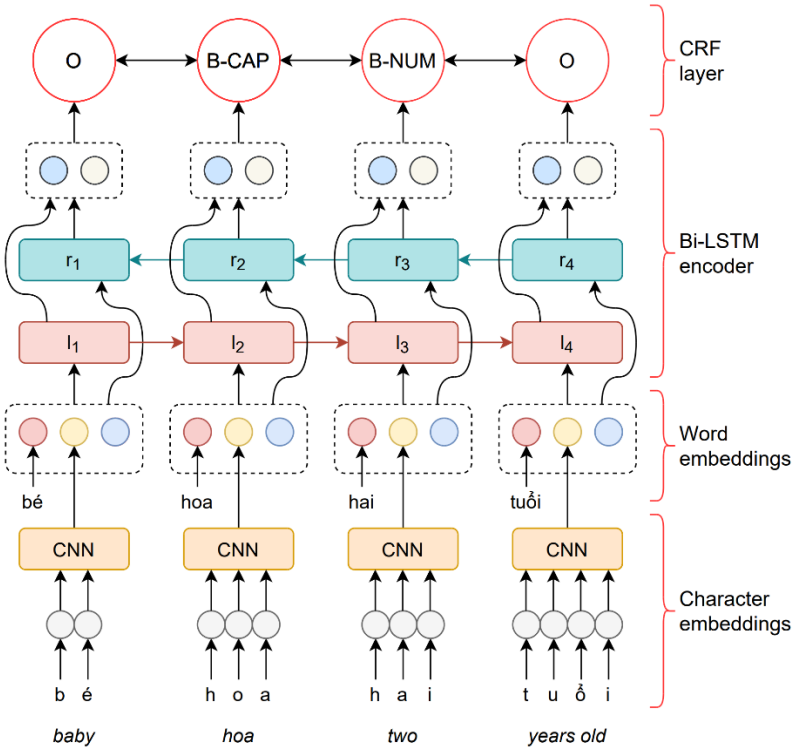


Figure 3.3.2. An architecture to detect entities in STT output using BiLSTMs.

3.4 Experiments

3.4.1 Corpus Building

This section introduces the steps we took to annotate the corpus. We first describe the annotation process, and then show some statistical figures on this corpus. The annotation process is illustrated in Figure 3.4.1. It includes the following four main steps:

(1) Collecting raw data: Texts from different sources, mainly from online newspapers, were collected.

(2) Pre-processing: Like other standard practices, the texts were split into sentences and the main punctuation was removed. During normalization, all words were converted to lowercases, and words with a dash or a colon were separated, keeping the dash and colon as words. Numbers/ dates/time/phone numbers/abbreviations/course numbers, etc., were transformed to their full, spoken forms. Then, we randomly selected 13K sentences and asked two annotators to manually label them with the required information.

(3) Label designing: We designed a corresponding set of labels that facilitates the goal of normalizing the output of STT systems. We developed guidelines containing these labels.

The guidelines provided detailed examples of the annotations, as well as specific information for each label that helped the annotators easier to annotate and solve conflicts in ambiguous cases. Three labels were chosen are Proper Noun, Dates/Time, and Numbers.

(4) Data tagging: To speed up the labeling step, a tool is built to automatically transform written texts into spoken forms of texts by using some rules such as:

- Phone-similar digit strings are transformed into spoken forms of each discrete digit.
- Some abbreviations are converted into their full forms by using a predefined list of Vietnamese abbreviations. This list is composed of scanning the newspapers to find out abbreviated words (they usually are not valid Vietnamese words). Then, a person is required to manually check and finalize the list.
- Long and short dates/time: We varied different reading methods of these dates/time. Popular reading methods are randomly chosen with higher probabilities.
- Numbers: like dates/time, we also diversify different reading methods for each number.

Then, we hire two annotators to manually check, and correct wrong labels and unnatural reading of a given text using the predefined set of labels designed in the previous step. To measure the inter-annotator agreement, we used Cohen's kappa coefficient [40]. Some statistics about the corpus are given in Table 3.4.1. Cohen's kappa coefficient of our corpus was 0.91, which usually is interpreted as almost perfect agreement.

No.	Entities	Number of Samples
1.	Proper Nouns	19.744
2.	Dates/Time	1.372
3.	Number	7.006
4.	Number of Sentences	13.000

Table 3.4.1. Some statistics about the corpus.

3.4.2 Experimental Setups

To create word embeddings, we collected the raw data from Vietnamese newspapers (~9GB texts) to train the word vector model using Glove[7] [41]. We fixed the number of word-embedding dimensions at 50, the number of character-embedding dimensions at 25. We also defined our own features and then mapped them into a vector of 10 dimensions.

For each experiment type, we conducted fivefold cross-validation tests. The hyper-parameters were chosen via a search on the development set. We randomly select 10% of the training data as the development set. The system performance is evaluated using precision, recall, and the F_1 score as in many sequence labeling problems [42] as follows:

$$F_1 = \frac{2 \cdot pre \cdot rec}{pre + rec}$$

$$pre = \frac{TP}{TP + FP}$$

$$rec = \frac{TP}{TP + FN}$$

where TP (True Positive) is the number of entities that are correctly identified. FP (False Positive) is the number of text sequences that are mistakenly identified as valid entities. FN (False Negative) is the number of entities that are not identified.

3.4.3 Experimental Results

In this section, we presented two types of experiments. The first one is to evaluate the effectiveness of the proposed method in detecting entities that need converting. This type includes three experiments to evaluate the baseline, the proposed method with/without using rich features. The second type is to integrate this text normalizer into a real STT system to measure its final performance on real output texts.

[7] https://github.com/standfordnlp/GloVe

3.4.3.1 Experimental Results of The Baseline Using Rules to Detect Entities

Based on some dictionaries about Vietnamese person names, location names, organization names, we designed some rules to captures these proper nouns automatically. For dates/time and numbers, we try to capture some popular reading ways of human beings among different regions of Vietnam. These rules are implemented by using the module *re* of Python language. Table 3.4.2 shows its experimental results. The baseline had a higher precision than recall in general because if a match is found it is probably correct. It got a precision of *82.64%*, a recall of *84.07%* and an F_1 score of *83.36%* averaged on five folds.

No.	Pre	Rec	F_1
Fold 1	82,66	84,21	83,43
Fold 2	82,69	84,15	83,41
Fold 3	82,47	83,83	83,14
Fold 4	82,84	84,18	83,50
Fold 5	82,55	84,06	83,30
Average	82,64	84,09	83,36

Table 3.4.2. Experiment results using the rule-based baseline to detect entities.

3.4.3.2 Experimental Results of the Proposed Model without Using Rich Features

Table 3.4.3 illustrates the experimental results of the proposed model. As can be seen that the proposed method got much higher results. The recall, precision, and F_1 scores are significantly increased on all five folds. Overall, it can greatly improve the efficiency of recognizing these entities. Specifically, compared to the baseline, this method remarkably boosted the F_1 metric by *4.11%*, precision by *6.9%*, and recall by *1.43%*.

No.	Pre	Rec	F_1
Fold 1	89,62	85,18	87,34
Fold 2	89,67	85,70	87,64
Fold 3	89,79	84,86	87,26

Fold 4	89,06	85,23	87,10
Fold 5	89,51	86,55	88,01
Average	89,53	85,50	87,47

Table 3.4.3. Experiment results using the neural architectures without rich features.

3.4.3.3 Experimental Results of the Proposed Model with Using Rich Features

Table 3.4.4 shows the experimental results using rich features. The results suggested that integrating rich features into neural models boosts the performance of the final system in detecting entities. In comparison to not using rich features, it increased the precision by *4.07%*, the recall by *2.42%*, and the F_1 score by *3.2%*, respectively.

No.	Pre	Rec	F_1
Fold 1	93,38	88,09	90,66
Fold 2	91,73	88,42	90,04
Fold 3	93,31	87,98	90,57
Fold 4	93,63	87,89	90,67
Fold 5	91,94	87,23	89,52
Average	92,80	87,92	90,29

Table 3.4.4. Experiment results using the neural architectures with rich features.

Figure 3.4.1 shows annotation process in building the new corpus and Figure 3.4.2 shows experimental results of the F_1 scores on each label. As can be seen that numbers are easiest to detect, followed by dates/time. Proper nouns are the most difficult to recognize because they are highly ambiguous.

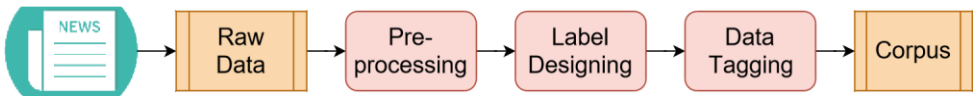


Figure 3.4.1. The annotation process in building the new corpus.

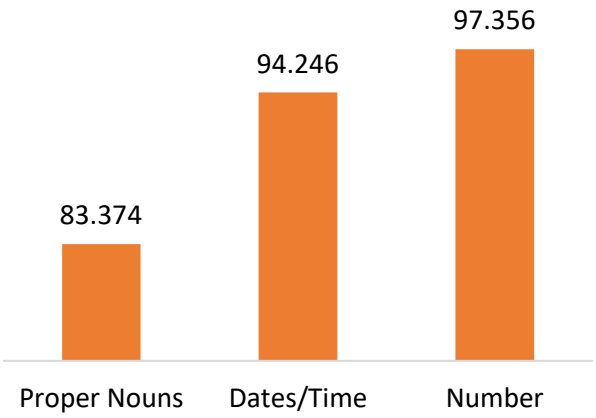


Figure 3.4.2. Performance on each label in the F_1 scores (in %).

3.4.3.4 Experimental Results of the Final System on the Output of a Real STT System

We integrated the best entity recognition model using biLSTM with rich features into our STT system to test its performance. The evaluation of the model was performed on 1000 real-world examples. Testers were required to read these randomly selected sentences as inputs and collect the outputs. Then, we also measured precision, recall, and F_1 scores based on the numbers of the entities such as proper nouns, dates/time, and numbers. The experimental results show that the text normalizer yields *75.78%* in precision, *82.2%* in recall, and *78.86%* in the F_1 score.

No.	Utterances	Wrong output of STT
1	Lý Hậu Lâm là giám đốc hãng hàng không Bamboo Airways *Ly Hau Lam is the director of Bamboo Airways*	Lý hận Lâm Lý hates Lâm
2	Chính quyền địa phương đã mời gia đình tới làm việc Authorities ask the family to collaborate	mười ten
3	Giải vô địch nữ quốc gia 2014 National champion award 2014	hai nghìn không trăm mười bốn (2000 trăm 14) two thousand zero hundred fourteen (2000 zero hundred 14)
4	Đó là chuyến làm khách tới Espanyol That is an away match to Espanyol	S Pa Non S Pa Non

Table 3.4.5. Some examples of wrong outputs of STT integrated with the text normalizer.

These results are lower than previous experiments on ideal spoken forms of texts. The main reason is that the real output of the STT produces some wrong words which make our text normalizer could not detect the sequence of texts that need converting. Some examples are shown in Table 3.4.5. The first case shows the wrong output of the middle name of a person. In the second case, the verb 'ask' was wrongly recognized as a number 'ten.' The third case shows an example of elision where a syllable 'zero' is dropped in the STT system. Observing the data, we saw several English words not correctly produced by STT system. This problem also causes a decrease in our text normalizer's performance as shown in the fourth case. There are also other types of errors which the entity recognizer could not detect out the right proper nouns, dates, and time.

3.5 Conclusion

We presented the first text normalization system for Vietnamese STT using deep neural architectures with rich features followed by manually designed rules. The neural architecture uses CNNs to encode character contexts of a word. Then, we concatenate them with pre-trained word embeddings and rich features to feed into a bi-LSTM encoder. A CRF is then applied on the top to predict label for each word. To conduct experiments, a newly built corpus is also presented for Vietnamese to facilitate the process of normalizing the output of STTs. This new dataset can serve as a benchmark for this task in Vietnamese. Experimental results on this corpus were promising. We achieved *90.67%* in the F_1 score in recognizing segments of texts that need normalizing, and *78.86%* in the F_1 score when tested on the real output of an STT system.

Through extensive experiments on this dataset, we acknowledge several insights such as using machine learning techniques is more robust and effective than rules in detecting entities in STT output texts, and some types of entities (e.g., numbers) are easier to detect than others (e.g., proper nouns). We hope that this initial study will inspire other follow-up research on this important but unexplored problem.

Chapter 4
Punctuation Restoration using Transformer-based Models

Punctuation prediction is the task of predicting and inserting punctuation like periods, commas, exclamation marks, etc. into the appropriate positions in transcribed texts in ASR systems. This helps to improve user readability and the performance of many downstream tasks. While most related studies have been performed for popular languages like English and Chinese, there is very little work done for low-resource languages. In order to stimulate the research on these languages, in this chapter, we target to improve the quality of punctuation prediction for Vietnamese ASRs. Specifically, we propose a method based on recent advances on pre-trained language models (LMs) for general purposes such as BERT and ELECTRA. The benefit of using these models is that they can be effectively fine-tuned on this punctuation prediction task where only a small amount of training data is available. To further enhance the performance, a simple yet effective technique to provide more context information in predicting punctuation marks for the very left and right words in each segment is also proposed. The experimental results of the proposed model on public benchmark datasets are quite promising. Overall, the proposed architecture substantially enhanced the prediction performance by a large margin and yielded a new state-of-the-art result on these datasets. Specifically, we achieved the F_1 scores of *71.49%* and *80.38%* on the Novel and Newspaper public datasets, respectively.

4.1 Introduction

Automatic Speech Recognition (ASR) systems normally generate un-normalized sequences of words (transcripts) which are difficult for human beings to read. It does not contain a proper segmentation into sentences as well as other predicted punctuation symbols like commas, question marks, etc. This also results in the degradation in performance of many further text processing tasks such as question answering, machine translation, etc. because these tasks are normally trained on punctuated texts [43] [44]. Hence, recovering punctuation marks is a very important step towards generating normalized texts [45] in the output of ASR systems. It relates to automatically infer the presence of punctuation and then insert it to the appropriate positions in the transcribed texts. Detecting punctuation in un-normalized texts requires quite a bit of linguistic sophistication and native speaker intuition [26]. Traditionally, this task was treated as a sequence labelling problem. Based on this approach, researchers exploited different machine learning

techniques with different kinds of features to enrich the prediction models. For example, there existed work which used raw speech waveforms with pause duration [46] [47] [48] [49], textual features only [50] [51] [52] [53], or both two feature types in a combination model to build the corresponding prediction models. Most of these studies have been done for popular languages such as Czech, English, French, German, Chinese, and Spanish [54].

While most work so far focused on high resource languages, there is very little work dedicated to low resource languages. In Vietnamese, the research about this field is still very limited. To our knowledge, there is only one work done for punctuator prediction in Vietnamese [55].

In recent years, we have witnessed an increasing interest in using pre-trained LMs based on transformers [7] to improve performances of many NLP tasks [1]. These models have been proposed to deal with the shortage of training data in many NLP tasks by pre-training LMs on many unlabeled datasets. These pre-trained models are then effectively fine-tuned on small, labeled datasets of the downstream NLP tasks. They typically result in substantial performance improvements compared to training on these small datasets from scratch [56]. For low-resource languages, this approach is highly effective when it is only possible to collect and annotate very little amount of labeled data for almost every NLP task. For Vietnamese, to the best of our knowledge, these models have not been investigated to address this punctuation prediction task. Hence, in this chapter, we aim at exploring these pre-trained LMs for this task in Vietnamese. Two kinds of models which are BERT [56] and ELECTRA [2] are exploited to boost the accuracy of predicting punctuation in Vietnamese transcribed texts.

In predicting punctuation, the whole transcribed text is usually long, so it is processed by splitting the text into the unit of segments before feeding into the model. For example, we acknowledged that the maximum segment length that BERT can process is 512 tokens. In such cases, the model is prone to make quite bad predictions for the words at the very left and right ends of the segment because there is not enough context information for those words at the boundary positions. To process such cases, we further propose a technique to provide more information about the surrounding words in making prediction of punctuation near those boundary words. This is called surrounding context information (or context information for short) hereafter.

In conclusion, the contribution of this chapter is three folds:

- Firstly, explore the effectiveness of pre-trained LMs for the task of punctuation prediction in a low-resource language, namely Vietnamese.
- Propose a simple yet effective technique to integrate surrounding contexts in predicting punctuation of the very left and right words in each segment.
- Extensively perform different kinds of experiments on public datasets to make comparisons and provide new SOTA results for future research in this interesting field.

The rest of this chapter is organized as follows. Section 2 discusses related work. In Section 3, we formally define the problem, and then propose a solution to solve it using pre-trained LMs. Section 4 describes the datasets, experimental setups, experimental results, and some discussions. Finally, we conclude the paper and point out some future lines of work in Section 5.

4.2 Related Work

So far, there have been many studies done for predicting punctuation in postprocessing the output of ASR systems. It can be divided into three main approaches based on the type of information available for building prediction models.

The first approach is to use the information of raw speech waveforms with pause duration between words, pitch, and intensity as inputs in order to make the prediction. And there is a significant variation in how different researchers use pauses [46] [47] [48]. However, such information is not always available.

In the case where audio is not provided, researchers have entirely relied on the appearance of texts only. Several methods have already been introduced to deal with this task by using textual information. The first method is to cast it as a sequence labeling problem and then exploit different machine learning algorithms such as conditional random fields (CRFs) [52] [53], deep and convolutional neural networks [51], or recently transformer-based models [50] to make punctuation prediction. Another method is to treat the punctuation restoration as a machine translation task, that is, translating from unpunctuated text to punctuated text [57]. There are also other methods which combine both feature types (prosodic features and text features) or build separate models then combined in various ways to further improve the performance.

In the case where the datasets are labeled with not only the information on punctuation marks but also the information on true casing of words, researchers proposed a variety of joint architectures to jointly learn both punctuation and true casing information in one go. For example, Sunkara et al., 2020 [58] proposed a joint framework using pre-trained masked LMs such as BERT to build the joint model. Nguyen et al., 2019 [59] proposed a method to restore the normalized texts for long-speech ASR transcription based on Transformer models and chunk merging.

As can be seen that most current work has been extensively studied for high resource languages like English, Spanish, Estonian or Chinese. These languages usually have many public benchmark datasets such as TED talks within IWSLT datasets[8], TDT4 English data[9], Wiki dataset [60], Tsinghua Chinese Treebank [61] and Estonian speech transcripts [49]. Unfortunately, there is very little work dedicated to low resource languages. In Vietnamese, the research about this field is still very limited. To our knowledge, there is only one work targeted to punctuator prediction [55]. Encouraging from the effectiveness of recent innovation in pre-trained LMs, in this chapter, we propose a learning architecture using BERT [1] and ELECTRA [2], to enhance the performance of punctuation prediction for Vietnamese ASRs. These models are quite effective especially for the NLP tasks with a small amount of labeled data [62]. Additionally, a technique to further boost the performance on predicting the punctuation of boundary words in each segment is also introduced.

4.3 A Proposed Model to Predict Punctuation

This section first formulates the problem and then introduces a proposed architecture based on pretrained LMs with extra boundary context information to solve the task.

4.3.1 Problem Definition

Given an input segment $S_{1:T} = \{x_1, x_2, \dots, x_T\}$, of length T and consisting of T syllables x_i. We assume that there are 6 classes corresponding to 6 possible punctuation marks mostly appear in

[8] https://sites.google.com/site/iwsltevaluation2016/

[9] http://ssli.ee.washington.edu/people/leixin/TDT4.html

the text documents which are *commas*, *periods*, *colons*, *question marks*, *exclamation marks*, and *semi-colons*.

We need to build a model f to map from each syllable x_i into its class $\hat{y}_i$ that maximize $P(\hat{y}_i|x_i)$ where $\hat{y}_i \in \{commas, periods, colons, question\text{-}marks, exclamation\text{-}marks, others\}$. The mapping indicates that the punctuation mark $\hat{y}_i$ is located right after the syllable x_i in the recovered form of this text segment. We add one more class, '*other*', to represent that there is no punctuation mark located right after x_i.

This problem can be considered as a sequence labelling task where we make a prediction label for each syllable in the input segment.

4.3.2 A proposed architecture using pre-trained LMs with boundary context information

We first describe the key points of pre-trained LMs, namely BERT [1] and ELECTRA [2]. Then, a proposed framework for learning and predicting punctuation using these LMs is presented. The reason for choosing these models is that many previous works have pointed out that these models normally yielded better performance on many downstream NLP tasks. In fact, the experimental results on the benchmark datasets also prove this statement.

BERT is trained based on Transformer [7]. Its attention mechanism learns contextual relations between words (or sub-words) in a text. It is basically an encoder-decoder model where an encoder reads the text input, and a decoder produces a prediction for the task. The goal of BERT is to generate a language model, therefore, only the encoder mechanism is necessary. Because the encoder reads the entire sequence of words at one time, it allows the model to learn the context of a word based on all its surroundings (left and right of the word). The detailed workings of Transformer can be found in the paper [1].

When training BERT, there are two prediction goals which are Masked Language Model (MLM) and Next Sentence Prediction (NSP) which are described in more details as follows:

- *MLM*: 15% of the words in each sequence are masked with a [MASK] token. BERT then learns to predict the original one of the masked words, based on the context provided by the other words which are non-masked in this sequence.

- *NSP*: BERT receives pairs of sentences as input and learns to predict if the second one is the subsequent sentence in the original document.

ELECTRA is trained to correctly predict typically 15% of masked tokens. Therefore, it can only learn from a small portion of text sequences. ELECTRA, which stands for Efficiently Learning an Encoder that Classifies Token Replacements, was proposed to deal with the disadvantage of BERT by learning the task called Replace Token Detection (RTD). It trains the model to distinguish between the real input token from the replaced one. Consequently, it is more effective in learning from the entire sequence instead of just a small portion of it. ELECTRA consists of two components as described below:

- *Generator*: is a small masked language model trained jointly with the discriminator. It is trained with maximum likelihood to predict masked words. After pre-training, this component is ignored and only the discriminator is fine-tuned on the punctuation prediction task.

- *Discriminator*: trains the model to distinguish between the real and the fake input data. Then, it is used to recognize which token has been replaced or kept the same.

The proposed architecture Figure 4.3.1 depicts our proposed architecture to solve the task. This architecture includes three main layers which are described as follows: Embedding Layer The input segment is tokenized by the WordPiece tokenizer [63] and fed into the BERT or ELECTRA encoders. Like the work of [55], we also trained the model on segments consisting of 100 syllables. However, for each segment we observe that the token in the middle positions is normally leveraged by the full context information of both left and right words in the input segment. While the very left or right words at the boundary of the segment only have very little or not enough surrounding context information at the left or right side, respectively. To provide more context for the words at these positions, more contexts are complemented to enrich the information for those words. Specifically, the sequences of k tokens that come before and after the segment are also included before feeding to the BERT/ELECTRA models.

The input segment enriched with the context information at the boundary is tokenized into $s_{1:T+2k} = \{x_{-k}, \dots, x_1, x_2, \dots, x_T, \dots, x_{T+k}\}$, of length $T + 2k$ and consisting of tokens x_i. When necessary, context information is padded to the left or to the right with [PAD] tokens.

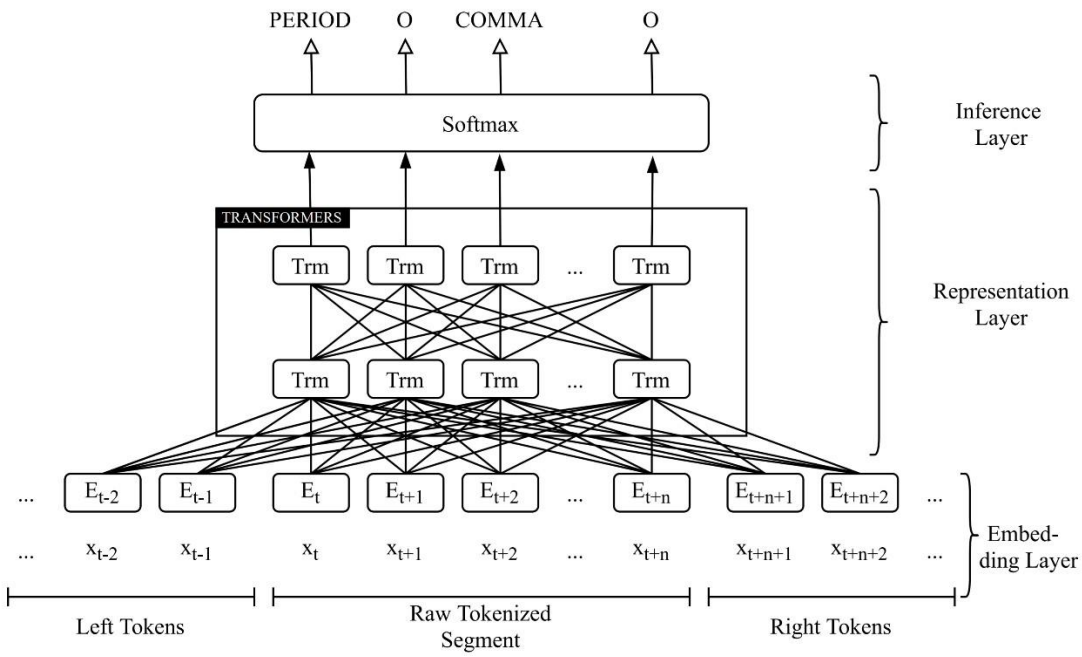


Figure 4.3.1. A proposed architecture using pre-trained LMs with the context information around each segment.

Representation Layer: The lower component exploits BERT or ELECTRA models to encode the context information of s into a representation vector $H = [h_{-k}, \ldots, h_{T+k}]$. After obtaining the representation, H, we design the decoder architecture on top of the BERT embedding layer for solving the task.

Inference Layer: This layer predicts a probability distribution over punctuation. In more details, the decoder takes in the vector representation h_i for token x_i and passes it through the activation layer L which is calculated as follows:

$$L_i = f(W_o h_i + b_o)$$

where f is a linear function, W_o is the weight matrix, and b_o is the bias vector of two layers. They are learnable parameters of the linear layer.

The prediction for this layer P is calculated using the corresponding sigmoid function which connects the activation layer with the output of this layer.

$$P(y_i \mid x_i) = softmax(L_i)$$

During training, the model minimizes the loss function of the predictor P over only the tokens of the original segment. We don't predict the labels for extra tokens added as surrounding contexts.

4.4 Experiments

4.4.1 Datasets

These two datasets (i.e., Novels and Newspaper) [55] were built from Vietnamese novels and newspapers. There are 6 types of punctuation marks with some statistics given in Table 4.4.1.

	Novel dataset				**News dataset**			
	Training set		**Testing set**		**Training set**		**Testing set**	
Punctuation	**Number**	**%**	**Number**	**%**	**Number**	**%**	**Number**	**%**
Comma (,)	50909	3.77	21231	4.045	482435	4.041	160472	4.054
Period (.)	66519	4.926	29643	5.648	419580	3.514	138967	3.51
Colon (:)	742	0.055	1153	0.221	32177	0.269	10728	0.271
Qmark (?)	14899	1.103	5271	1.004	13902	0.116	4468	0.113
Exclam (!)	30183	2.235	9167	1.747	7384	0.062	2333	0.059
Semicolon (;)	48	0.004	43	0.008	5675	0.048	2045	0.052
Sentences	**111601**		**44081**		**440866**		**145768**	

Table 4.4.1. Some statistics about the two Vietnamese benchmark datasets which are Novel and News.

These datasets are divided into training, validation and testing with the ratio of 6:2:2. In these datasets, there is no assumption about sentence boundaries inside texts. We also noticed that the distribution of punctuation marks is not equal. Some marks (i.e., *comma*, *period*) dominate, while the others (i.e., colon, semicolon) only appear very little.

4.4.2 Experimental Setups

For BERT, we exploited two variants which are mBERT[10] and viBERT[11]. For ELECTRA, we exploited the vELECTRA[12] which is pre-trained for the Vietnamese language. The viBERT and vELECTRA [56] are optimally pre-trained LMs for the Vietnamese language.

Our models were implemented in PyTorch[13]. We set the batch size to 32 for both viBERT and vELECTRA. The maximum sequence length is set at 100 which is like the previous work [55]. The value of k was set to 50. Our models were trained using the AdamW optimizer [64], a stochastic optimization method that modifies the typical implementation of weight decay in the Adam optimizer by decoupling weight decay from the gradient update. The learning rate was tuned in [1e-5, 1.5e-5, 2e-5, 3e-5, 5e-5, 1e-4] with a linear warm up schedule. To reduce overfitting, we also added dropout with a dropout rate of 0.1 for BERT components and 0.5 for the last layer of fine-tuning.

All experiments were performed on the server where its hardware components are CPU Intel(R) Xeon(R) CPU E5-2698 v4 @ 2.20GHz, and GPU NVIDIA Tesla V100 32GB CoWoS HBM2 PCIe 3.0.

4.4.3 Evaluation Metrics

The system performance is evaluated using precision, recall, and the F_1 score as in many sequence labeling problems as follows:

---

[10] https://github.com/google-research/bert/blob/master/multilingual.md

[11] https://github.com/fpt-corp/viBERT

[12] https://github.com/fpt-corp/vELECTRA

[13] https://pytorch.org/

$$F_1 = \frac{2 \cdot pre \cdot rec}{pre + rec}$$

$$pre = \frac{TP}{TP + FP}$$

$$rec = \frac{TP}{TP + FN}$$

where TP (True Positive) is the number of punctuation marks that are correctly identified. FP (False Positive) is the number of punctuation marks that are mistakenly identified as valid ones. FN (False Negative) is the number of punctuation marks that are not identified.

4.4.4 Experimental Results

This section presents extensive experiments to prove the effectiveness of the proposed architecture and some discussions on the final experimental results.

Firstly, we show experimental results of the proposed architecture with and without using surrounding context information. The results on the NOVEL dataset are illustrated first, followed by the results on the Newspaper dataset. Additionally, to see the impact of the pre-trained models on this downstream task, we performed experiments with different training data sizes to see the performance of the final model on the two datasets. Finally, some discussions on comparison with the previous work and performance on each punctuation mark are also presented.

4.4.4.1 Experimental results on the NOVEL dataset

Table 4.4.2 and Table 4.4.3 show experimental results of the proposed models on the NOVEL dataset with and without using surrounding contexts. Among three models, the vELECTRA yielded the best F_1 scores and outperformed the other two models by a large margin on all three-evaluation metrics. Using vELECTRA, we achieved the F_1 scores of *69.56%* without using surrounding contexts, and *71.49%* by using surrounding contexts. In comparison with mBERT, the one optimized for Vietnamese, viBERT, boosted the F_1 scores by *4.3%* and *3.8%* with and without using surrounding contexts, respectively.

Punctuation	mBERT			viBERT			vELECTRA		
	Pre	Rec	F_1	Pre	Rec	F_1	Pre	Rec	F_1
PERIOD	62.77	64.58	63.66	66.32	68.97	67.62	72.05	73.63	72.83
COMMA	61.21	52.58	56.57	63.31	59.67	61.43	68.44	64.48	66.40
COLON	26.21	2.34	4.30	36.84	4.25	7.62	48.39	5.20	9.40
SCOLON	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
QMARK	77.34	73.17	75.20	77.77	75.81	76.78	80.72	79.53	80.12
EMARK	59.24	58.44	58.84	61.48	60.83	61.15	62.88	66.02	64.41
MICROAVG.	62.91	59.47	61.14	65.57	64.25	64.90	70.24	68.89	**69.56**

Table 4.4.2. Experimental results on the NOVEL dataset WITHOUT using surrounding contexts.

Punctuation	mBERT			viBERT			vELECTRA		
	Pre	Rec	F_1	Pre	Rec	F_1	Pre	Rec	F_1
PERIOD	64.45	66.33	65.37	69.21	70.90	70.05	73.28	76.46	74.84
COMMA	62.88	54.70	58.51	66.76	61.24	63.88	71.18	66.06	68.53
COLON	25.76	2.95	5.29	43.20	4.68	8.45	47.85	6.76	11.85
SCOLON	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
QMARK	79.18	75.20	77.14	80.27	77.63	78.93	84.11	80.72	82.38
EMARK	59.70	62.27	60.96	62.61	63.90	63.25	64.08	66.96	65.49
MICROAVG.	64.36	61.62	62.96	68.33	66.19	67.24	72.07	70.91	71.49

Table 4.4.3. Experimental results on the NOVEL dataset using surrounding contexts.

4.4.4.2 Experimental results on the NEWSPAPER dataset

Table 4.4.4 and Table 4.4.5 show experimental results of the proposed models on the NEWSPAPER dataset with and without using surrounding contexts. Observing the results, we draw the same conclusion as shown on the Novel dataset. Specifically, the vELECTRA outperformed the other two models and yielded new state-of-the-art results. We achieved *78.30%* and *80.38%* in the F_1 scores without and with using surrounding information, respectively. Because viBERT is optimized on Vietnamese language, this specialized model gave a higher improvement compared to mBERT. It boosts the F_1 scores by *2.8%* and *3%* with and without using surrounding contexts, respectively.

Punctuation	mBERT			viBERT			vELECTRA		
	Pre	Rec	F_1	Pre	Rec	F_1	Pre	Rec	F_1
PERIOD	78.60	80.27	79.43	80.05	84.35	82.14	83.85	86.01	84.92
COMMA	69.92	67.16	68.51	74.51	68.98	71.64	75.46	74.27	74.86
COLON	58.66	47.48	52.48	62.34	48.77	54.73	65.48	56.34	60.56
SCOLON	31.54	12.03	17.42	30.59	19.46	23.79	44.28	10.22	16.61
QMARK	64.86	58.12	61.31	68.81	63.94	66.29	70.12	70.01	70.06
EMARK	38.07	11.36	17.50	38.63	12.30	18.66	38.14	16.67	23.20
MICROAVG.	73.34	71.32	72.32	76.45	74.19	75.31	78.71	77.89	78.30

Table 4.4.4. Experimental results on the NEWSPAPER dataset WITHOUT surrounding contexts.

Punctuation	mBERT			viBERT			vELECTRA		
	Pre	Rec	F_1	Pre	Rec	F_1	Pre	Rec	F_1
PERIOD	80.68	83.12	81.88	82.17	86.67	84.36	85.75	89.03	87.36
COMMA	71.68	69.28	70.46	75.52	71.59	73.50	77.99	74.93	76.43
COLON	62.27	50.14	55.55	66.62	50.57	57.50	69.47	57.87	63.14
SCOLON	34.11	16.53	22.27	33.49	17.11	22.65	42.11	21.12	28.13
QMARK	68.32	62.76	65.42	68.99	71.04	70.00	75.73	72.22	73.94
EMARK	41.60	12.73	19.49	45.83	10.59	17.20	49.21	13.29	20.92
MICROAVG.	75.31	73.82	74.56	78.11	76.65	77.37	81.12	79.66	80.38

Table 4.4.5. Experimental results on the NEWSPAPER dataset using surrounding contexts.

4.4.4.3 Experimental results with different training sizes

For a better understanding of the performance of the proposed architecture, we performed an empirical comparison taking into consideration the level of different sizes of training sets. On each dataset, we varied the training data sizes by *10%* and performed this 10 times. Figure 4.4.1 and Figure 4.4.2 shows experimental results using the best model, vELECTRA, with surrounding contexts on the Novel and Newspaper datasets.

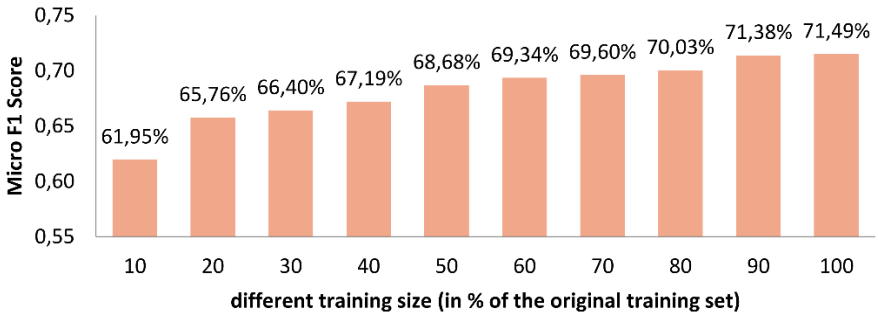


Figure 4.4.1. The F_1 scores using different training sizes using vELECTRA with context information on the NOVEL dataset.

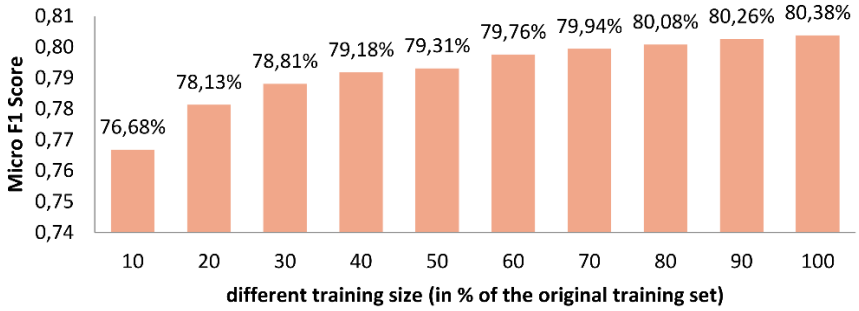


Figure 4.4.2. The F_1 scores using different training sizes using vELECTRA with context information on the NEWSPAPER dataset.

4.4.5 Discussion

4.4.5.1 Comparison with the best baseline model

Table 5.4.5-1 shows experimental results on the two datasets of the best baseline model (i.e., biLSTM without focal losses) and the best proposed model (i.e., vELECTRA) integrated with surrounding contexts. We observed a significant improvement of the proposed model over the baseline one mentioned in [55] on all three-evaluation metrics of precision, recall and F_1 scores. Specifically, on the Novel dataset, vELECTRA boosted the F_1 score by *17.71%*. On the Newspaper dataset, it boosted the F_1 score by *21.18%*.

Datasets	Models	Pre	Rec	F_1
NEWSPAPER	biLSTM-attention (BAW)	69.63	56.97	62.67
	vELECTRA	**81.12**	**79.66**	**80.38**
NOVEL	biLSTM-attention (BAW)	56.52	45.34	50.31
	vELECTRA	**72.07**	**70.91**	**71.49**

Table 4.4.6. Experimental results of the best baseline method (biLSTM) and the best proposed model (vELECTRA) integrated with surrounding contexts (micro averaged on all 6 punctuation marks).

As we can see, the more training data we have, the better the performance of the model. However, with only very little amount of data, we still achieve quite good performance. For example, with only *10%* of original training data size, we could obtain around *62%* and *76.7%* in the F_1 scores on the Novel and Newspaper datasets, respectively. These results set new SOTA results for future work for this task using these benchmark datasets.

4.4.5.2 Performance on each punctuation mark

Table 4.4.2, Table 4.4.3, Table 4.4.4, and Table 4.4.5 also show the experimental results of the proposed model on all punctuation marks. We observed that the proposed models yielded relatively high performance on three labels which are *period*, *comma*, and *question marks*. On the Novel dataset, the performance on two labels of *colon* and *semi-colon* are lowest. On the News dataset, the performance on the two labels of *exclamation* and *semi-colon* are lowest. The reason is that the number of samples in the training datasets on these labels are much smaller in comparison to other labels. This imbalance data problem needs solving in order to improve the performance on these skewed labels.

4.5 Conclusion

This chapter presents an effort to improve the performance of a punctuation prediction model on Vietnamese ASRs. To cope with the data sparsity problem in low-resource languages, we proposed to exploit the recent innovation of pretrained LMs to solve the task. Trained on a large amount of unlabeled data for general purposes, these pre-trained LMs can be further fine-tuned effectively on many downstream NLP tasks with a small amount of training data. In more detail, we used two LMs, namely BERT and ELECTRA, which are specifically pre-trained for the

Vietnamese language. To improve even more, we also proposed a technique to better make the prediction of punctuation marks of the boundary syllables in each segment. The experimental results of these proposed models on two public datasets showed promising results. The best model integrated with surrounding contexts yielded the best performance and established new challenging SOTA results. It outperformed the best baseline model by a large margin. Using the best model with surrounding contexts, we achieved *71.49%* and *80.38%* in the F_1 scores on the Novel and Newspaper datasets, respectively.

In the future, we will continue to consider the imbalance data problem to deal with the skewed data of rare punctuation marks.

Chapter 5
Conclusion and Future Work

5.1 Conclusion

This study presented the text normalization system for Vietnamese STT using deep neural architectures with rich features, surrounding contexts and Vietnamese transformer-based pretrained language models. The neural architecture uses BERT or ELECTRA to encode contexts of sequences. Then, we concatenate them rich features or surrounding contexts to feed into encoder layers. To conduct experiments, a newly built corpus is also presented for Vietnamese to facilitate the process of normalizing the output of STTs. This new dataset can serve as a benchmark for this task in Vietnamese. Experimental results on this corpus were promising.

5.2 Future Work

Through extensive experiments on this dataset, we acknowledge several insights such as using machine learning techniques is more robust and effective than rules in detecting entities in STT output texts, and some types of entities (e.g., numbers) are easier to detect than others (e.g., proper nouns). We hope that this initial study will inspire other follow-up research on this important but unexplored problem.

We will continue to investigate the efficacy of transformer-based language models with various model architectures, such as RoBERTa or XLM, in the future. Furthermore, hierarchical models that combine the spoken-form entity job with the punctuation restoration goal show great promise. We are considering creating a new nested dataset that combines these two objectives into one nested task due to a paucity of low-resource language text normalization experiments. We hope that this work will spur more research into this essential yet understudied issue.

References

[1] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, 2019.

[2] K. Clark, M.-T. Luong, Q. V. Le and C. D. Manning, "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.

[3] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee and L. Zettlemoyer, "Deep Contextualized Word Representations," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, 2018.

[4] A. Radford and K. Narasimhan, "Improving Language Understanding by Generative Pre-Training," 2018.

[5] Z. Yang, Z. Dai, Y. Yang, J. G. Carbonell, R. Salakhutdinov and Q. V. Le, "XLNet: Generalized Autoregressive Pretraining for Language Understanding," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, 2019.

[6] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *CoRR*, vol. abs/1907.11692, 2019.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention is All you Need," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, 2017.

[8] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[9] P. Le-Hong, A. Roussanaly, N. T. M. Huyên and M. Rossignol, "An empirical study of maximum entropy approach for part-of-speech tagging of Vietnamese texts," in *Actes de la 17e conférence sur le Traitement Automatique des Langues Naturelles. Articles longs, TALN 2010, Montréal, Canada, July 2010*, 2010.

[10] D. Q. Nguyen, D. Q. Nguyen, D. D. Pham and S. B. Pham, "RDRPOSTagger: A Ripple Down Rules-based Part-Of-Speech Tagger," in *Proceedings of the 14th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2014, April 26-30, 2014, Gothenburg, Sweden*, 2014.

[11] X. Ma and E. H. Hovy, "End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*, 2016.

[12] T. Vu, D. Q. Nguyen, D. Q. Nguyen, M. Dras and M. Johnson, "VnCoreNLP: A Vietnamese Natural Language Processing Toolkit," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 2-4, 2018, Demonstrations*, 2018.

[13] D. Q. Nguyen, "A neural joint model for Vietnamese word segmentation, POS tagging and dependency parsing," in *Proceedings of the The 17th Annual Workshop of the Australasian*

Language Technology Association, ALTA 2019, Sydney, Australia, December 4-6, 2019, 2019.

[14] D. Q. Nguyen and A. T. Nguyen, "PhoBERT: Pre-trained language models for Vietnamese," in *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, 2020.

[15] P. Le-Hong, "Vietnamese Named Entity Recognition using Token Regular Expressions and Bidirectional Inference," *CoRR*, vol. abs/1610.05652, 2016.

[16] V.-T. Luong and L. K. Pham, "ZA-NER: Vietnamese Named Entity Recognition at VLSP 2018 Evaluation Campaign," *In Proceedings of Vietnamese Speech and Language Processing (VLSP)*, 2018.

[17] T.-H. Pham and P. Le-Hong, "End-to-End Recurrent Neural Network Models for Vietnamese Named Entity Recognition: Word-Level Vs. Character-Level," in *Computational Linguistics - 15th International Conference of the Pacific Association for Computational Linguistics, PACLING 2017, Yangon, Myanmar, August 16-18, 2017, Revised Selected Papers*, 2017.

[18] P. Q. N. Minh, "A Feature-Based Model for Nested Named-Entity Recognition at VLSP-2018 NER Evaluation Campaign," *CoRR*, vol. abs/1803.08463, 2018.

[19] P. Q. N. Minh, "A Feature-Rich Vietnamese Named-Entity Recognition Model," *CoRR*, vol. abs/1803.04375, 2018.

[20] T.-H. Pham and P. Le-Hong, "The Importance of Automatic Syntactic Features in Vietnamese Named Entity Recognition," in *Proceedings of the 31st Pacific Asia Conference on Language, Information and Computation, PACLIC 2018, Cebu City, Philippines, November 16-18, 2017*, 2017.

[21] H. Pham, K. Pham, T.-A. Nguyen and L.-H. Phuong, "NNVLP: A Neural Network-Based Vietnamese Language Processing Toolkit," 2017.

[22] H. Nguyen, Q. Ngo, L. Vu, V. Tran and H. Nguyen, "VLSP Shared Task: Named Entity Recognition," *Journal of Computer Science and Cybernetics*, vol. 34, pp. 283-294, January 2019.

[23] X.-S. Vu, T. Vu, S. N. Tran and L. Jiang, "ETNLP: A Visual-Aided Systematic Approach to Select Pre-Trained Embeddings for a Downstream Task," in *Proceedings of the International Conference on Recent Advances in Natural Language Processing, RANLP 2019, Varna, Bulgaria, September 2-4, 2019*, 2019.

[24] T. C. Luong and P. Le-Hong, "Towards Task-Oriented Dialogue in Mixed Domains," in *Computational Linguistics - 16th International Conference of the Pacific Association for Computational Linguistics, PACLING 2019, Hanoi, Vietnam, October 11-13, 2019, Revised Selected Papers*, 2019.

[25] P. Le-Hong, N. T. M. Huyên, T. L. Nguyen and M.-L. Ha, "Fast Dependency Parsing Using Distributed Word Representations," in *Trends and Applications in Knowledge Discovery and Data Mining - PAKDD 2015 Workshops: BigPMA, VLSP, QIMIE, DAEBH, Ho Chi Minh City, Vietnam, May 19-21, 2015. Revised Selected Papers*, 2015.

[26] H. Schütze, Ambiguity resolution in language learning - computational and cognitive models, vol. 71, CSLI, 1997.

[27] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, pp. 1735-1780, November 1997.

[28] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami and C. Dyer, "Neural Architectures for Named Entity Recognition," in *NAACL HLT 2016, The 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, San Diego California, USA, June 12-17, 2016*, 2016.

[29] Y. LeCun, B. E. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. E. Hubbard and L. D. Jackel, "Handwritten Digit Recognition with a Back-Propagation Network," in

Advances in Neural Information Processing Systems 2, [NIPS Conference, Denver, Colorado, USA, November 27-30, 1989], 1989.

[30] G. Eryigit and D. Torunoglu-Selamet, "Social media text normalization for Turkish," *Nat. Lang. Eng.*, vol. 23, p. 835–875, 2017.

[31] T. Ikeda, H. Shindo and Y. Matsumoto, "Japanese Text Normalization with Encoder-Decoder Model," in *Proceedings of the 2nd Workshop on Noisy User-generated Text, NUT@COLING 2016, Osaka, Japan, December 11, 2016*, 2016.

[32] H. Hassan and A. Menezes, "Social Text Normalization using Contextual Graph Random Walks," in *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics, ACL 2013, 4-9 August 2013, Sofia, Bulgaria, Volume 1: Long Papers*, 2013.

[33] A. Aw, M. Zhang, J. Xiao and J. Su, "A Phrase-Based Statistical Model for SMS Text Normalization," in *ACL 2006, 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, Sydney, Australia, 17-21 July 2006*, 2006.

[34] S. Yolchuyeva, G. Németh and B. Gyires-Tóth, "Text normalization with convolutional neural networks," *Int. J. Speech Technol.*, vol. 21, p. 589–600, 2018.

[35] J. D. Lafferty, A. McCallum and F. C. N. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," in *Proceedings of the Eighteenth International Conference on Machine Learning (ICML 2001), Williams College, Williamstown, MA, USA, June 28 - July 1, 2001*, 2001.

[36] N. X. Bach, N. D. Linh and T. M. Phuong, "An empirical study on POS tagging for Vietnamese social media text," *Comput. Speech Lang.*, vol. 50, p. 1–15, 2018.

[37] O. T. Tran and T. C. Luong, "Towards Understanding User Requests in AI Bots," in *PRICAI 2018: Trends in Artificial Intelligence - 15th Pacific Rim International Conference on Artificial Intelligence, Nanjing, China, August 28-31, 2018, Proceedings, Part I*, 2018.

[38] Q. Zhu, X. Li, A. Conesa and C. Pereira, "GRAM-CNN: a deep learning approach with local context for named entity recognition in biomedical text," *Bioinform.*, vol. 34, p. 1547–1554, 2018.

[39] T. Mikolov, S. Kombrink, L. Burget, J. Cernocký and S. Khudanpur, "Extensions of recurrent neural network language model," in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing, ICASSP 2011, May 22-27, 2011, Prague Congress Center, Prague, Czech Republic*, 2011.

[40] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, pp. 37-46, 1960.

[41] J. Pennington, R. Socher and C. D. Manning, "Glove: Global Vectors for Word Representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, 2014.

[42] V. Yadav and S. Bethard, "A Survey on Recent Advances in Named Entity Recognition from Deep Learning models," in *Proceedings of the 27th International Conference on Computational Linguistics, COLING 2018, Santa Fe, New Mexico, USA, August 20-26, 2018*, 2018.

[43] N. X. Bach and T. M. Phuong, "Leveraging User Ratings for Resource-poor Sentiment Classification," in *19th International Conference in Knowledge Based and Intelligent Information and Engineering Systems, KES 2015, Singapore, 7-9 September 2015*, 2015.

[44] O. T. Tran, N. X. Bach, M. L. Nguyen and A. Shimazu, "Answering Legal Questions by Mining Reference Information," in *New Frontiers in Artificial Intelligence - JSAI-isAI 2013 Workshops, LENLS, JURISIN, MiMI, AAA, and DDS, Kanagawa, Japan, October 27-28, 2013, Revised Selected Papers*, 2013.

[45] O. T. Tran and V. T. Bui, "Neural Text Normalization in Speech-to-Text Systems with Rich Features," *Appl. Artif. Intell.*, vol. 35, p. 193–205, 2021.

[46] H. Christensen, Y. Gotoh and S. Renals, "Punctuation annotation using statistical prosody models.," 2001.

[47] M. Igras, B. Ziólko, P. Zelasko and M. Witkowski, "Structure of pauses in speech in the context of speaker verification and classification of speech type," *EURASIP J. Audio Speech Music. Process.*, vol. 2016, p. 18, 2016.

[48] T. Levy, V. Silber-Varod and A. Moyal, "The effect of pitch, intensity and pause duration in punctuation detection," in *2012 IEEE 27th Convention of Electrical and Electronics Engineers in Israel*, 2012.

[49] O. Tilk and T. Alumäe, "Bidirectional Recurrent Neural Network with Attention Mechanism for Punctuation Restoration," in *Interspeech 2016, 17th Annual Conference of the International Speech Communication Association, San Francisco, CA, USA, September 8-12, 2016*, 2016.

[50] T. Alam, A. Khan and F. Alam, "Punctuation Restoration using Transformer Models for High-and Low-Resource Languages," in *Proceedings of the Sixth Workshop on Noisy User-generated Text, W-NUT@EMNLP 2020 Online, November 19, 2020*, 2020.

[51] X. Che, C. Wang, H. Yang and C. Meinel, "Punctuation Prediction for Unsegmented Transcript Based on Word Vector," in *Proceedings of the Tenth International Conference on Language Resources and Evaluation LREC 2016, Portorož, Slovenia, May 23-28, 2016*, 2016.

[52] W. Lu and H. T. Ng, "Better Punctuation Prediction with Dynamic Conditional Random Fields," in *Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing, EMNLP 2010, 9-11 October 2010, MIT Stata Center, Massachusetts, USA, A meeting of SIGDAT, a Special Interest Group of the ACL*, 2010.

[53] N. Ueffing, M. Bisani and P. Vozila, "Improved models for automatic punctuation prediction for spoken and written text," in *INTERSPEECH 2013, 14th Annual Conference*

of the International Speech Communication Association, Lyon, France, August 25-29, 2013, 2013.

[54] M. Ballesteros and L. Wanner, "A Neural Network Architecture for Multilingual Punctuation Generation," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, 2016.

[55] T. Pham, N. Nguyen, Q. Pham, H. Cao and B. T. Nguyen, "Vietnamese Punctuation Prediction Using Deep Neural Networks," in *SOFSEM 2020: Theory and Practice of Computer Science - 46th International Conference on Current Trends in Theory and Practice of Informatics, SOFSEM 2020, Limassol, Cyprus, January 20-24, 2020, Proceedings*, 2020.

[56] T. V. Bui, T. O. Tran and P. Le-Hong, "Improving Sequence Tagging for Vietnamese Text using Transformer-based Neural Models," in *Proceedings of the 34th Pacific Asia Conference on Language, Information and Computation, PACLIC 2020, Hanoi, Vietnam, October 24-26, 2020*, 2020.

[57] E. Cho, J. Niehues and A. Waibel, "NMT-Based Segmentation and Punctuation Insertion for Real-Time Spoken Language Translation," in *Interspeech 2017, 18th Annual Conference of the International Speech Communication Association, Stockholm, Sweden, August 20-24, 2017*, 2017.

[58] M. Sunkara, S. Ronanki, K. Dixit, S. Bodapati and K. Kirchhoff, "Robust Prediction of Punctuation and Truecasing for Medical ASR," *CoRR*, vol. abs/2007.02025, 2020.

[59] B. Nguyen, V. B. H. Nguyen, H. Nguyen, P. N. Phuong, T.-L. Nguyen, Q. T. Do and L. C. Mai, "Fast and Accurate Capitalization and Punctuation for Automatic Speech Recognition Using Transformer and Chunk Merging," in *22nd Conference of the Oriental COCOSDA International Committee for the Co-ordination and Standardisation of Speech Databases and Assessment Techniques, O-COCOSDA 2019, Cebu, Philippines, October 25-27, 2019*, 2019.

[60] R. Sproat and N. Jaitly, "RNN Approaches to Text Normalization: A Challenge," *CoRR*, vol. abs/1611.00068, 2016.

[61] Y. Zhao, C. Wang and G. Fu, "A CRF Sequence Labeling Approach to Chinese Punctuation Prediction," in *Proceedings of the 26th Pacific Asia Conference on Language, Information and Computation, PACLIC 26, Bali, Indonesia, December 16-18, 2012*, 2012.

[62] O. T. Tran and V. T. Bui, "A BERT-based Hierarchical Model for Vietnamese Aspect Based Sentiment Analysis," in *12th International Conference on Knowledge and Systems Engineering, KSE 2020, Can Tho City, Vietnam, November 12-14, 2020*, 2020.

[63] R. Sennrich, B. Haddow and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*, 2016.

[64] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.

[65] K. A. Nguyen, N. Dong and C.-T. Nguyen, "Attentive Neural Network for Named Entity Recognition in Vietnamese," in *2019 IEEE-RIVF International Conference on Computing and Communication Technologies, RIVF 2019, Danang, Vietnam, March 20-22, 2019*, 2019.

[66] D. Q. Nguyen, T. Vu, D. Q. Nguyen, M. Dras and M. Johnson, "From Word Segmentation to POS Tagging for Vietnamese," in *Proceedings of the Australasian Language Technology Association Workshop, ALTA 2017, Brisbane, Australia, December 6-8, 2017*, 2017.